Course: COMP 333 — Final Project Phase 1 and 2

Group: O
Names:

- Carson Johnston - **Student ID:** 40312846
- Charlotte Lauzon - **Student ID:** 40285642
- Ava Samimi - **Student ID:** 40048117

Description of the task:

The objective of Phase 1 of this project is to select a large, real-world dataset (>= 1GB). We then do the data retrieval, the wrangling, an EDA with 2 research questions, a baseline model and a report on Jupyter Notebook.

Phase 2 then implements advanced machine learning algorithms, engineers informative features, and applies unsupervised learning techniques. In this Phase, we do comparative analysis and model interpretation. This is a big step in answering our Research Questions.

Source of the datasets:

The **US Used Cars** dataset is a public dataset available on Kaggle:
https://www.kaggle.com/datasets/ananaymital/us-used-cars-dataset

Description of the datasets:

The **US Used Cars** dataset exceeds the minimum size requirements, has a large number of features, includes real-world inconsistencies and supports both supervised and unsupervised learning tasks.

It contains detailed information about 3 million used cars details across the United States.
Each row represents a single vehicle listing, wth each column being an attribute.

The *US Used Cars* dataset contains:

- Vehicle Identification (Vehicle identification number, listing ID)
- Vehicle Specifications (Engines, horsepower, fuel, size, legroom, bed, body...)
- Market/Dealer Information (city, ZIP, position, days on market, date of listing)
- Fuel efficiency
- Condition and History (accidents, damages, if it was a cab, if it is new...)
- Categorical and Descriptive Features (colours, description, picture...)

The dataset has a mix of numerical, boolean, high-cardinality categorical and string variables

**The notebook will include: **

1. Data Retrieval
2. Wrangling/Cleaning
3. EDA
4. Baseline Model
5. Report

Division-of-labour The work was split and shared by all team members. More precisely:
Carson: phase 1 Step 2 (Wrangling/Cleaning), with contributions to phase 1 step 3 (Research question) and phase 1 step 4 (evaluation, discussion). Phase 2 step 2 (Feature Engineering) with contributions to phase 2 step 4 (Interpretation, feature importance, partial dependence plots, insights)
Charlotte: Phase 1 Step 1 (Data Retrieval), with contributions to phase 1 step 2 (cleaning pipeline), phase 1 step 3 (Research question), phase 1 step 4 (setup of the model, train/val/test, linear regression, metrics and evaluation setup) and the overall introduction and README.md. Phase 2 Step 1 (Advanced Supervised Learning), with contributions to Phase 2 step 4 (Interpretation, feature importance, relation to research question)
Ava: Pase 1 Step 3 (EDA), with contributions to phase 1 step 4 (discussion) and README.md. Phase 2 step 3 (Unsupervised learning)

Please run this next cell for any Phase to import the required libraries!

```
#import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import matplotlib.gridspec as gridspec
from pandas.api.types import is_numeric_dtype, is_bool_dtype
import warnings
import os
import re

# Avoid unnecessary warnings in output
#warnings.filterwarnings("ignore")

# Doesn't cut tables
pd.set_option('display.width', 1000)
pd.set_option('display.max_colwidth', 50)
pd.set_option('display.max_columns', 30)

# Apply predefined visual theme for consistence in the export
plt.style.use('seaborn-v0_8')
sns.set_theme()
```

Phase 1

1. Data Retrieval: Phase 1 – Step 1

1.1 Source

We are using the **US Used Cars** public dataset from Kaggle: https://www.kaggle.com/datasets/ananaymital/us-used-cars-dataset

This dataset is downloaded as a zipped csv file, and, extracted, has a size of approximately 9.98 GB.
It contains tabular vehicle listing data.

1.2 Retrieval

The dataset is retrieved as a **file-based CSV source** and is loaded into Python using the pandas function `read_csv()`.

We selected file-based retrieval, because the data is structured and tabular, and Kaggle provides it as a flat CSV file.
Also, by retrieving the data as a CSV file, it avoids API authentification dependency. Since the dataset is static and doesn't change in real time, the most reproducible and efficient approach is to store the raw file locally.

1.3 Challenge Handling

- **Large file size (~10GB):** Because the Data file is nearly 10 GB, direct loading can be slow and memory-heavy.
    - Mitigation:
        - We made a sample of the data of ~1GB so the full trimmed data would be used
            - We read the original file by chunks
            - Sampling was performed uniformly across all chunks of the dataset to ensure that all portions of the data were represented.
        - we use `low_memory=False`. This makes sure the entire file is read before deciding the columns' data type. This prevents dtype warnings.
- **Data type Inconsistencies:** Some numeric variables are stored as strings; During the Data Wrangling Phase, these will be cleaned and converted.
- **Authentication / Rate Limits:** Not applicable in this phase because there are no API calls used (download is done once via Kaggle).
    - If Kaggle API were used, authentication would require a private Kaggle API token. However, for this phase, one static file download was more efficient and safe. To ensure grading reproducibility, we avoid credential-based retrieval,.

1.4 Raw Data Storage

The dataset is stored in the project's root directory and is treated as read-only to preserve the data integrity. All transformations and cleaning steps are performed on the copied dataset.

```
# DO NOT LOAD THIS UNLESS YOU HAVE A VERY POWERFUL COMPUTER
# This is to load the full 9.98GB dataset
'''
raw_data_path = "used_cars_data.csv"

# Load the raw dataset
if os.path.exists(raw_data_path):
    print("Loading full dataset...")
    cars = pd.read_csv(raw_data_path, low_memory=False)
else:
    print("Full dataset not found.")

print("Shape:", cars.shape)
cars.head()
'''
```

```
'\nraw_data_path = "used_cars_data.csv"\n\n# Load the raw dataset\nif os.path.exists
(raw_data_path):\n    print("Loading full dataset...")\n    cars = pd.read_csv
(raw_data_path, low_memory=False)\nelse:\n    print("Full dataset not found.")\n\nprint
("Shape:", cars.shape)\ncars.head()\n'
```

This block here reads the full input file by chunks of 200k rows, then samples each chunk to create a trim dataset of approximately 1.2 GB

After creating the trimmed dataset, it is loaded as dataframe

```
# STEP 1: Create trimmed raw dataset (skipped if clean_cars.csv is already present)
input_file = "used_cars_data.csv"
output_file = "used_cars_trimmed_raw.csv"
chunksize = 200_000
sample_frac = 0.12

if os.path.exists(input_file):
    # Remove old trimmed file if it already exists
    if os.path.exists(output_file):
        os.remove(output_file)

    first_write = True

    print("Creating trimmed raw dataset iteratively...")

    for chunk_idx, chunk in enumerate(pd.read_csv(input_file, chunksize=chunksize,
low_memory=False)):
        # Sample rows from each chunk without cleaning or changing columns
        sampled_chunk = chunk.sample(frac=sample_frac, random_state=42)

        # Write first chunk with header, append remaining chunks without header
        sampled_chunk.to_csv(
            output_file,
            mode="w" if first_write else "a",
            header=first_write,
            index=False
        )

        first_write = False
        print(f"Processed chunk {chunk_idx + 1}")

    print("Trimmed raw dataset saved as:", output_file)

else:
    print("Full dataset not found. Skipping trimmed dataset creation.")
```

```
Creating trimmed raw dataset iteratively...
Processed chunk 1
Processed chunk 2
Processed chunk 3
Processed chunk 4
Processed chunk 5
Processed chunk 6
Processed chunk 7
Processed chunk 8
Processed chunk 9
Processed chunk 10
Processed chunk 11
Processed chunk 12
Processed chunk 13
Processed chunk 14
Processed chunk 15
Processed chunk 16
Trimmed raw dataset saved as: used_cars_trimmed_raw.csv
```

If the "used_cars_trimmed_raw.csv" file is in the project's root directory, you can simply run this following cell

```
# STEP 2: Load dataset into testing_cars
trimmed_path = "used_cars_trimmed_raw.csv"
if os.path.exists(trimmed_path):
    print("Loading trimmed raw dataset...")
    testing_cars = pd.read_csv(trimmed_path, low_memory=False)
else:
    raise FileNotFoundError(
        "Trimmed dataset not found. Please create used_cars_trimmed_raw.csv from
used_cars_data.csv (see README)."
    )

print("Shape:", testing_cars.shape)
testing_cars.head()
```

```
Loading trimmed raw dataset...
Shape: (360005, 66)
```

	vin	back_legroom	bed	bed_height	bed_length	body_type	cabin	cit
0	JTMDFREV2HJ173436	37.2 in	NaN	NaN	NaN	SUV / Crossover	NaN	North Attlebor
1	5TDGZRBH1LS518759	41 in	NaN	NaN	NaN	SUV / Crossover	NaN	Wobur
2	3FA6P0HD3HR375170	38.3 in	NaN	NaN	NaN	Sedan	NaN	Lapee
3	2HGFC3B37HH355203	35.9 in	NaN	NaN	NaN	Coupe	NaN	Forest Hill
4	19XFC2F57HE215572	37.4 in	NaN	NaN	NaN	Sedan	NaN	Windsor Lock

5 rows × 66 columns

2. Wrangling/Cleaning: Phase 1 – Step 2

We perform an initial data audit:

- Missing value analysis
- Duplicate detection
- Outlier detection
- Data type validation
- Reproducible cleaning pipeline

We use the `quantDDA()` and `vizDDA()` functions from Lab 2 to standardize our audit process.

2.1 INITIAL AUDIT

```
# The function quantDDA(df) takes a pandas DataFrame as argument
# The function returns a new DataFrame with a summary of each column
def quantDDA(df):

# create a list to put all column's summary in
    summary_list = []

# Loop through all columns of the argument dataset
    for column in df.columns:
        # One column is a series, we create a dictionary to store the statistics
        series = df[column]
        summary = {}

        # Store the column name as the feature
        summary["Feature"] = column
        # Store the number of rows of the database
        summary["Number of Observations"] = len(series)
        # Store the number of only valid, non-missing values
        summary["Number of Entries"] = series.count()
        # Store the number of distinct values only
        summary["Number of Unique Entries"] = series.nunique()
        # Store the number of missing values (where its n/a is "true")
        summary["Number of Missing Entries"] = series.isna().sum()

        # Store the mode values as a list
        modes = series.mode()
        summary["Mode(s)"] = list(modes)

        # The categorical columns do not have all the numeric fields;
        # To make sure the table has no issues with columns, we set as NaN
        summary["Number of Outliers (IQR)"] = np.nan
        summary["Number of Extreme Values (top/bottom 1%)"] = np.nan
        summary["Mean"] = np.nan
        summary["Standard Deviation"] = np.nan
```

```
        summary["Maximum"] = np.nan
        summary["Minimum"] = np.nan
        summary["Q1"] = np.nan
        summary["Q2 (Median)"] = np.nan
        summary["Q3"] = np.nan
        summary["Skewness"] = np.nan
        summary["Kurtosis"] = np.nan

        # If its numerical column, compute numeric statistics
        # Otherwise, skip the calculations
        if pd.api.types.is_numeric_dtype(series) and not pd.api.types.is_bool_dtype
(series):
            # We remove missing values to not compute with invalid numbers
            # pandas ignore NaN, but not SciPy, so we drop beforehand
            x = series.dropna()

            # Store all the numerical statistics
            if len(x) > 0:
                summary["Mean"] = x.mean()
                summary["Standard Deviation"] = x.std()
                summary["Maximum"] = x.max()
                summary["Minimum"] = x.min()
                summary["Q1"] = x.quantile(0.25)
                summary["Q2 (Median)"] = x.median()
                summary["Q3"] = x.quantile(0.75)

                # Number of outliers with IQR Method
                IQR = summary["Q3"] - summary["Q1"]
                lower = summary["Q1"] - 1.5 * IQR
                upper = summary["Q3"] + 1.5 * IQR
                # Count values outside the lower and upper bounds
                outliers = x[(x < lower) | (x > upper)]
                summary["Number of Outliers (IQR)"] = outliers.count()

                # Extreme values (Top and Bottom 1%)
                lower_ext = x.quantile(0.01)
                upper_ext = x.quantile(0.99)
                # Counts the number of extreme values of the top and bottom 1%
                extreme = x[(x < lower_ext) | (x > upper_ext)]
                summary["Number of Extreme Values (top/bottom 1%)"] = extreme.count()

                # Use SciPy for skewness and kurtosis
                summary["Skewness"] = stats.skew(x, bias=False)
                summary["Kurtosis"] = stats.kurtosis(x, fisher=True, bias=False)

        # Add the dictionary to list
        summary_list.append(summary)

    result = pd.DataFrame(summary_list)

    # Make sure the columns are in the same order as in assignment
    ordered_columns = [
        "Feature",
```

```
        "Number of Observations",
        "Number of Entries",
        "Number of Unique Entries",
        "Number of Missing Entries",
        "Number of Outliers (IQR)",
        "Number of Extreme Values (top/bottom 1%)",
        "Mode(s)",
        "Mean",
        "Standard Deviation",
        "Maximum",
        "Minimum",
        "Q1",
        "Q2 (Median)",
        "Q3",
        "Skewness",
        "Kurtosis"
    ]

    return result[ordered_columns]
```

```
# The function vizDDA(df) takes a pandas DataFrame as argument
# The function produces a square grid of plots (univariate on diagonal, bivariate off
diagonal),
# and a heatmap of the missing values.
def vizDDA(df):

    df = df.copy()

# Sample if too large to avoid slow crosstabs and memory issues
    max_rows = 5000
    if len(df) > max_rows:
        df = df.sample(n=max_rows, random_state=42)

    # Remove high-cardinality categorical columns for visualization grid
    high_cardinality_cols = []

    for col in df.columns:
        if not pd.api.types.is_numeric_dtype(df[col]):
            # put an arbitrary 20 threshold
            if df[col].nunique() > 20:
                high_cardinality_cols.append(col)
    # df for the plots
    df_plot = df.drop(columns=high_cardinality_cols)

    # Limit grid size to avoid memory/speed issues (max 6x6)
    max_features = 6
    if len(df_plot.columns) > max_features:
        df_plot = df_plot.iloc[:, :max_features]
    num_features = len(df_plot.columns)

    # Detect datetime columns
    is_datetime = {
```

```
        column: pd.api.types.is_datetime64_any_dtype(df_plot[column])
        for column in df_plot.columns
    }

    # Create square grid
    # put 4x4 inch space for each subplot for readability
    fig = plt.figure(figsize=(4 * num_features, 4 * num_features))
    grid = gridspec.GridSpec(num_features, num_features)

    # Consider i as index for rows, and j as index for columns
    for i, col_i in enumerate(df_plot.columns):
        for j, col_j in enumerate(df_plot.columns):

            ax = fig.add_subplot(grid[i, j])

            # Detect if its numeric or categorical
            is_num_i = pd.api.types.is_numeric_dtype(df_plot[col_i])
            is_num_j = pd.api.types.is_numeric_dtype(df_plot[col_j])

            # Univariate on diagonal
            # if we compare a feature with itself, we are doing a univariate analysis
            if i == j:

                # If it's a datetime univariate, we count how many times each unique
datetime appear
                if is_datetime[col_i]:
                    series = df_plot[col_i].dropna()
                    if len(series) > 0:
                        series.value_counts().sort_index().plot(ax=ax)
                    ax.set_ylabel("count")

                # if numeric, we do a histogram to visualize the distribution
                elif is_num_i:
                    sns.histplot(df_plot[col_i], kde=True, ax=ax)

                # For categorical univariate, we do a bar chart
                else:
                    df_plot[col_i].value_counts().plot(kind="bar", ax=ax)

            # Bivariate off diagonal
            # We study two different variables
            else:

                # If one variable is datetime, and the other is numeric:
                # We do a line plot
                # Datetime vs Numeric when datetime is the row variable, and numeric is
column variable
                if is_datetime[col_i] and is_num_j:
                    # Select the relevant columns, drop the missing values, and sort by
the datetime column
                    tmp = df_plot[[col_i, col_j]].dropna().sort_values(col_i)
                    # We draw datetime on x-axis, numeric on y-axis
                    ax.plot(tmp[col_i], tmp[col_j])
```

```
                # Same but for the symmetric situation;
                # Datetime vs Numeric when datetime is the column variable, and numeric
is row variable
                elif is_datetime[col_j] and is_num_i:
                    tmp = df_plot[[col_j, col_i]].dropna().sort_values(col_j)
                    ax.plot(tmp[col_j], tmp[col_i])

                # If both variables are numeric (Numeric vs numeric), we do a scatter plot
                elif is_num_i and is_num_j:
                    sns.scatterplot(x=df_plot[col_j], y=df_plot[col_i], ax=ax)

                # If there are mixed types (categorical vs numeric) we do a boxplot
                # For when numerical is the row feature and categorical is the column
feature
                elif is_num_i and not is_num_j and not is_datetime[col_j]:
                    tmp = df_plot[[col_j, col_i]].dropna()
                    tmp[col_j] = tmp[col_j].astype("category")
                    sns.boxplot(data=tmp, x=col_j, y=col_i, ax=ax)

                # For when numerical is the column feature and categorical is the row
feature
                elif not is_num_i and is_num_j and not is_datetime[col_i]:
                    tmp = df_plot[[col_i, col_j]].dropna()
                    tmp[col_i] = tmp[col_i].astype("category")
                    sns.boxplot(data=tmp, x=col_i, y=col_j, ax=ax)

                # If both variable are categorical (Categorical vs categorical), we do a
grouped bar chart
                # Since all cases with numerical variables have been checked,
                # only categorical on both row and colum left
                else:
                    categorical = pd.crosstab(df_plot[col_i], df_plot[col_j])
                    categorical.plot(kind="bar", ax=ax)

            # Axis labeling
            # Only label bottom row with x labels
            if i == num_features - 1:
                ax.set_xlabel(col_j, fontsize=15, fontweight="bold")
            else:
                ax.set_xlabel("")

            # Only label left column with y labels
            if j == 0:
                ax.set_ylabel(col_i, fontsize=15, fontweight="bold")
            else:
                ax.set_ylabel("")

    # Add title
    fig.suptitle("Visualization Grid (Univariate on Diagonal, Bivariate Off-Diagonal)",
fontsize=25, fontweight="bold")
    # Improve spacing so subplots don't overlap
    plt.tight_layout(rect=[0, 0, 1, 0.96])
```

```
    plt.show()

    # Heatmap of missing values
    plt.figure(figsize=(12, 6))
    sns.heatmap(df.isna(), cbar=True, vmin=0, vmax=1)
    plt.title("Missing Values Heatmap")
    plt.show()
```

General Findings of the First Audit

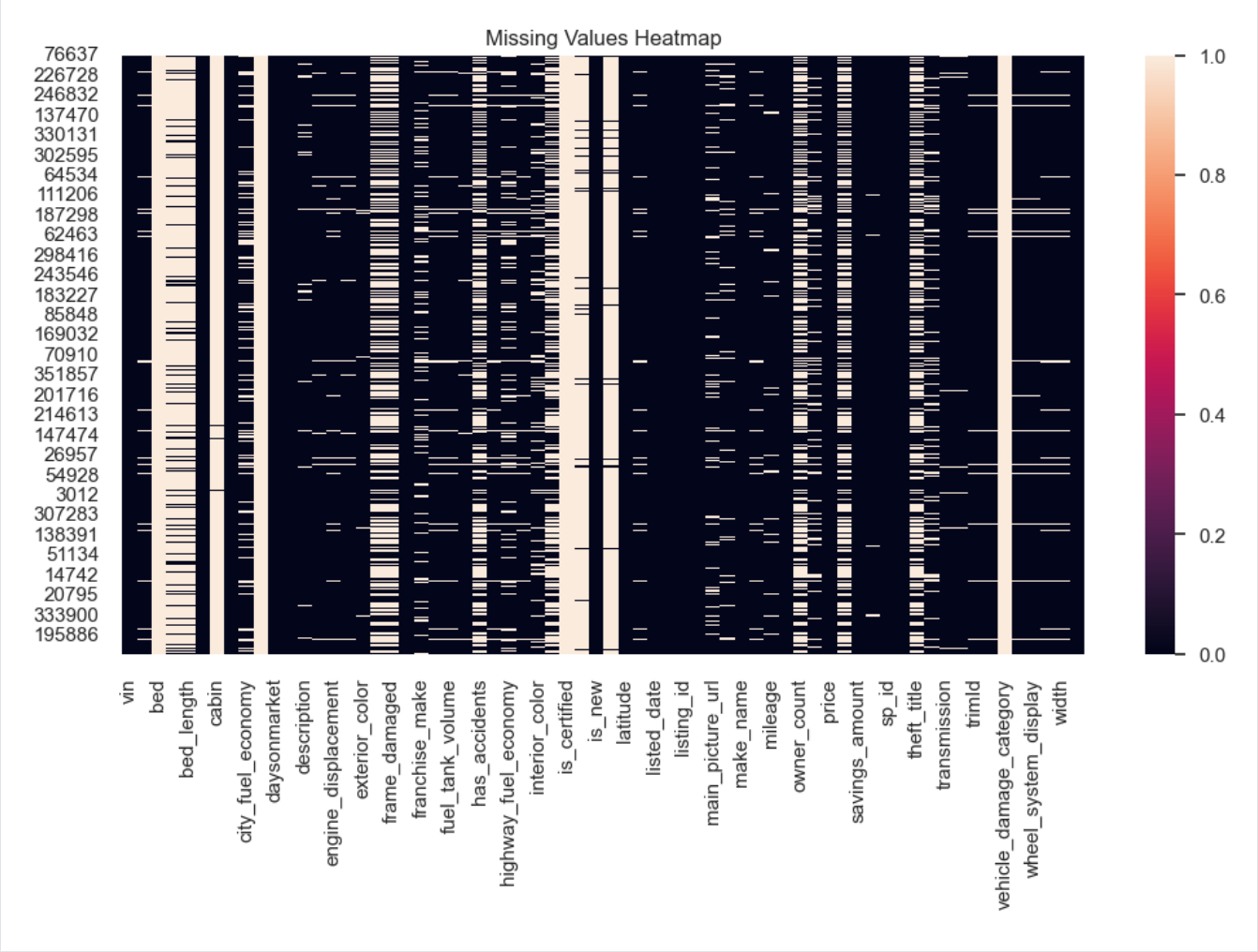
```
quantDDA(testing_cars)
```

	Feature	Number of Observations	Number of Entries	Number of Unique Entries	Number of Missing Entries	Number of Outliers (IQR)	Number of Extreme Values (top /bottom 1%)	M
0	vin	360005	360005	360004	0	NaN	NaN	[1FT7W2BT8GEC
1	back_legroom	360005	340854	200	19151	NaN	NaN	[3
2	bed	360005	2354	3	357651	NaN	NaN	
3	bed_height	360005	51657	1	308348	NaN	NaN	
4	bed_length	360005	51657	78	308348	NaN	NaN	[6
...	...	...	...	...	...	...	...	
61	wheel_system	360005	342438	5	17567	NaN	NaN	[
62	wheel_system_display	360005	342438	5	17567	NaN	NaN	[Front-Wheel
63	wheelbase	360005	340854	452	19151	NaN	NaN	[10
64	width	360005	340854	274	19151	NaN	NaN	[7
65	year	360005	360005	87	0	32469.0	2816.0	

66 rows × 17 columns

```
vizDDA(testing_cars)
```

Visualization Grid (Univariate on Diagonal, Bivariate Off-Diagonal)



2.2: Reproducible Pipeline

For the cleaning pipeline, we first manually remove columns. Then, we apply a function to clean the dataset

A cleaning Report is provided for an easier view of what was performed.

Manual Removal of Columns

Due to time contraints, computing costs and the scope of this project. Below is a method that manually removes columns from the large dataframe. The columns selected to be removed were chosen by doing data analysis with functions found below such as clean_cars(). The reason remove_cars_columns() is implemented so early on is to speed up the computing times of processes below. This function only works on the "used_cars_data.csv" file used for this assignment. It will not work with other data sets.

The idea of this is to reduce the amount of columns in the data frame so more rows can be loaded in for better data accuracy and better performance

The manual removal of columns is meant to compliment the reproducible cleaning pipeline. It is not a replacement and will not remove columns that our pipline does automatically.

Removal of Copy Columns

This process involves finding columns that share similar data and removing them. For example "wheel_system" and "wheel_system_display" both describe if cars are front wheel drive (FWD) or all wheel drive (AWD). "wheel_system_display" is not needed and will be removed from the data frame.

Removal of Descriptive Columns

This process involves removing columns that focus on describing car features, rather than giving simple data. It is hard to process descriptive data, so columns such as "description" will be removed.

Removal of Unecessary Columns

Remove columns that don't add much value to the data analysis. These columns are removed to allow for more rows from more useful columns. 'trimId" is an example of a column that doesn't provide much usefulness.

```
def remove_cars_columns(df):
    copy_columns = ['wheel_system_display', 'engine_cylinders', 'latitude', 'longitude',
'exterior_color', 'listing_id', 'power']
    descriptive_columns = ['description', 'trim_name', 'transmission_display', 'sp_name',
'major_options']
    unecessary_columns = ['trimId', 'dealer_zip', 'savings_amount', 'sp_id',
'seller_rating']


    remove_columns = []
    remove_columns.extend(copy_columns)
    remove_columns.extend(descriptive_columns)
    remove_columns.extend(unecessary_columns)

    return df.drop(columns=[c for c in remove_columns if c in df.columns], errors='ignore'
)

testing_cars = remove_cars_columns(testing_cars)
```

Cleaning of the Data Set

Reproducible Data Cleaning Pipeline

The clean_cars() function implements a structured and reproducible data cleaning pipeline. Compared to the manual removal process described earlier, this function performs automated transformations based on measurable data properties (e.g., missingness rate, data type detection, regex pattern matching). This function ensures that cleaning steps are systematic, transparent, and reproducible.

Defensive Copy of the Dataset

A copy of the dataframe is created to prevent accidental modification of the original dataset. This preserves the raw data as read-only.

Dropping Columns with Extremely High Missingness

This step computes the proportion of missing values per column, removes columns where the missing rate exceeds a user-defined threshold, and stores removed columns in a cleaning report. We do this because columns with excessive missing values add noise, increase computational cost, and provide unreliable signal for modeling.

Removing Duplicate Rows

This step removes fully duplicated rows, because duplicates can bias statistical results, artificially inflate dataset size, and distort frequency distributions.

The number of removed rows is tracked in the report for transparency.

Handling Missing Values (Numeric)

For numeric columns, missing values are imputed using a missing indicator column and the median. This preserves information about missingness via a flag, and median imputation is robust to outliers.

Handling Missing Values (Categorical)

For categorical columns, missing values are replaced with a label "Missing". This preserves row count, avoids dropping potentially useful data, and allows models to treat the missingness as a category.

String Unification and Measurement Normalization

We standardize string formatting by converting to string, removing the whitespace and detecting measurements columns. For example, we checked if a column contained a pattern such as "20 in", "15.5 in"...

If the pattern was detected, we extracted only the numeric portion, and converted to float. We also renamed to column to reflect the units.

```
#Create function for pipeline
def clean_cars(df, drop_missing_threshold):
    df = df.copy()
```

```
    report = {}

    # Drop columns with extremely high missingness
    missing_rate = df.isna().mean()
    drop_cols = missing_rate[missing_rate > drop_missing_threshold].index.tolist()
    df = df.drop(columns=drop_cols)
    report["dropped_cols_high_missing"] = drop_cols

    # Remove row duplicates
    before = len(df)
    df = df.drop_duplicates()
    report["rows_removed_duplicates"] = before - len(df)

    # Missing values
    # Numeric columns: add missing flag
    imputed_numeric = []
    for col in df.columns:
        if is_numeric_dtype(df[col]) and not is_bool_dtype(df[col]):
            if df[col].isna().any():
                df[f"{col}_missing"] = df[col].isna()
                df[col] = df[col].fillna(df[col].median())
                imputed_numeric.append(col)
    report["numeric_imputed_with_flags"] = imputed_numeric

    # For categorical columns: fill missing with label
    filled_categorical = []
    for col in df.columns:
        if df[col].dtype == "object" and df[col].isna().any():
            df[col] = df[col].fillna("Missing")
            filled_categorical.append(col)
    report["categorical_filled"] = filled_categorical

    # String unification & converting inches measurements into numeric values
    for col in df.select_dtypes(include=["object", "string"]).columns:
        # Standardize strings first
        df[col] = df[col].astype(str).str.strip()

        # Check if the column mostly contains "number in" patterns
        # We'll check the first non-null value as a heuristic
        sample_value = df[col].dropna().iloc[0] if not df[col].empty else ""

        # Regex pattern: look for digits (and optional decimal) followed by 'in'
        if re.search(r'\d+\.?\d*\s*in', str(sample_value).lower()):
            # 1. Extract the number
            df[col] = df[col].str.extract(r'(\d+\.?\d*)').astype(float)

            # 2. Rename the column (e.g., 'wheel_size' -> 'wheel_size_in')
            df = df.rename(columns={col: f"{col}_in"})

    # column "fuel_tank_volume" will be converted to numerical data
    # Ensure the column is treated as a string
    column_as_string = df['fuel_tank_volume'].astype(str)
```

```
    # Extract the numbers (including decimals) using Regex
    # (\d+\.?\d*) looks for digits, an optional dot, and more digits
    df['fuel_tank_volume'] = column_as_string.str.extract(r'(\d+\.?\d*)').astype(float)

    # Rename the column to keep track of the units
    df = df.rename(columns={'fuel_tank_volume': 'fuel_tank_gal'})

    # column "maximum_seating" will be converted to numerical data
    # ensure teh column is treated as a string
    column_as_string = df['maximum_seating'].astype(str)

    # extract the numbers using Regex
    # (\d+\.?\d*) looks for digits, an optional dot, and more digits. I have no idea if
vehicles can have half seats
    df['maximum_seating'] = column_as_string.str.extract(r'(\d+\.?\d*)').astype(float)

    return df, report
```

Cleaning Report

```
cars_clean, cleaning_report = clean_cars(testing_cars, 0.1)
cleaning_report
```

```
{'dropped_cols_high_missing': ['bed',
  'bed_height',
  'bed_length',
  'cabin',
  'city_fuel_economy',
  'combine_fuel_economy',
  'fleet',
  'frame_damaged',
  'franchise_make',
  'has_accidents',
  'highway_fuel_economy',
  'interior_color',
  'isCab',
  'is_certified',
  'is_cpo',
  'is_oemcpo',
  'main_picture_url',
  'owner_count',
  'salvage',
  'theft_title',
  'torque',
  'vehicle_damage_category'],
 'rows_removed_duplicates': 1,
 'numeric_imputed_with_flags': ['engine_displacement',
  'horsepower',
  'mileage'],
 'categorical_filled': []}
```

Preview of the Cleaned Dataset

```
cars_clean.head(10)
```

	vin	back_legroom_in	body_type	city	daysonmarket	engine_displace
0	JTMDFREV2HJ173436	37.2	SUV / Crossover	North Attleboro	28	25
1	5TDGZRBH1LS518759	41.0	SUV / Crossover	Woburn	46	35
2	3FA6P0HD3HR375170	38.3	Sedan	Lapeer	78	20
3	2HGFC3B37HH355203	35.9	Coupe	Forest Hills	187	15
4	19XFC2F57HE215572	37.4	Sedan	Windsor Locks	7	20
5	5TFDY5F12LX951920	42.3	Pickup Truck	Huntington Station	27	57
6	5TDJZRFH9JS529988	38.4	SUV / Crossover	Schenectady	7	35
7	3GKALVEX0LL193793	39.7	SUV / Crossover	Vernon Rockville	69	20
8	1HGCR2F34HA146733	38.5	Sedan	Abington	5	24
9	KMHH35LEXLU121366	34.8	Hatchback	Stamford	336	20

Summary Statistics for the cleaned dataset

Removal of outliers

The function to remove outliers will only be implemented at the end of the EDA and data cleaning. This is to prevent functions like quantDDA from doing analysis on data that has outliers removed.

The process of removing outliers relies on simple algorithms like the one found in quantDDA, but also requires a human understanding of the data columns. For example, the quantDDA function might identify 0 as an outlier for the "mileage" column, but it is very reasonable for a new car to have not driven.

For the outlier removal function a list will be created for columns that require different observations. Just below is the list in markdown format with reasoning for special treatment.

- maximum_seating: Between 2 and 15 seats is reasonable for small cars and vans
- mileage: Cars with 0 mileage are possible if they are brand new. Only upper outliers will be removed
- year: Only exclude the lower end of outliers. Cars made the year the data was collected are not outliers.

IMPORTANT: only use this function once data cleaning is final and you are ready to move on to proper analysis.

```
# remove outliers specific to used cars dataset
def remove_car_outliers(df):
    # list of columns to completly ignore, just add column names to ignore for testing
    columns_ignore = ['maximum_seating']

    for column in df.columns:
        series = df[column]
        # only compute numeric types
        if pd.api.types.is_numeric_dtype(series) and not pd.api.types.is_bool_dtype
(series):
            # only act on columns that need outliers removed
            if column not in columns_ignore:

                summary = {}

                Q1 = series.quantile(0.25)
                Q3 = series.quantile(0.75)
                IQR = Q3 - Q1

                lower = Q1 - 1.5 * IQR
                upper = Q3 + 1.5 * IQR

                # only remove lower outliers for year
                if column == 'year':
                    df = df[df[column] >= lower]
                elif column == 'mileage':
                    df = df[df[column] <= upper]
                else:
                    # Remove both upper and lower outliers for any other included numeric
columns
                    df = df[(df[column] >= lower) & (df[column] <= upper)]

    return df
```

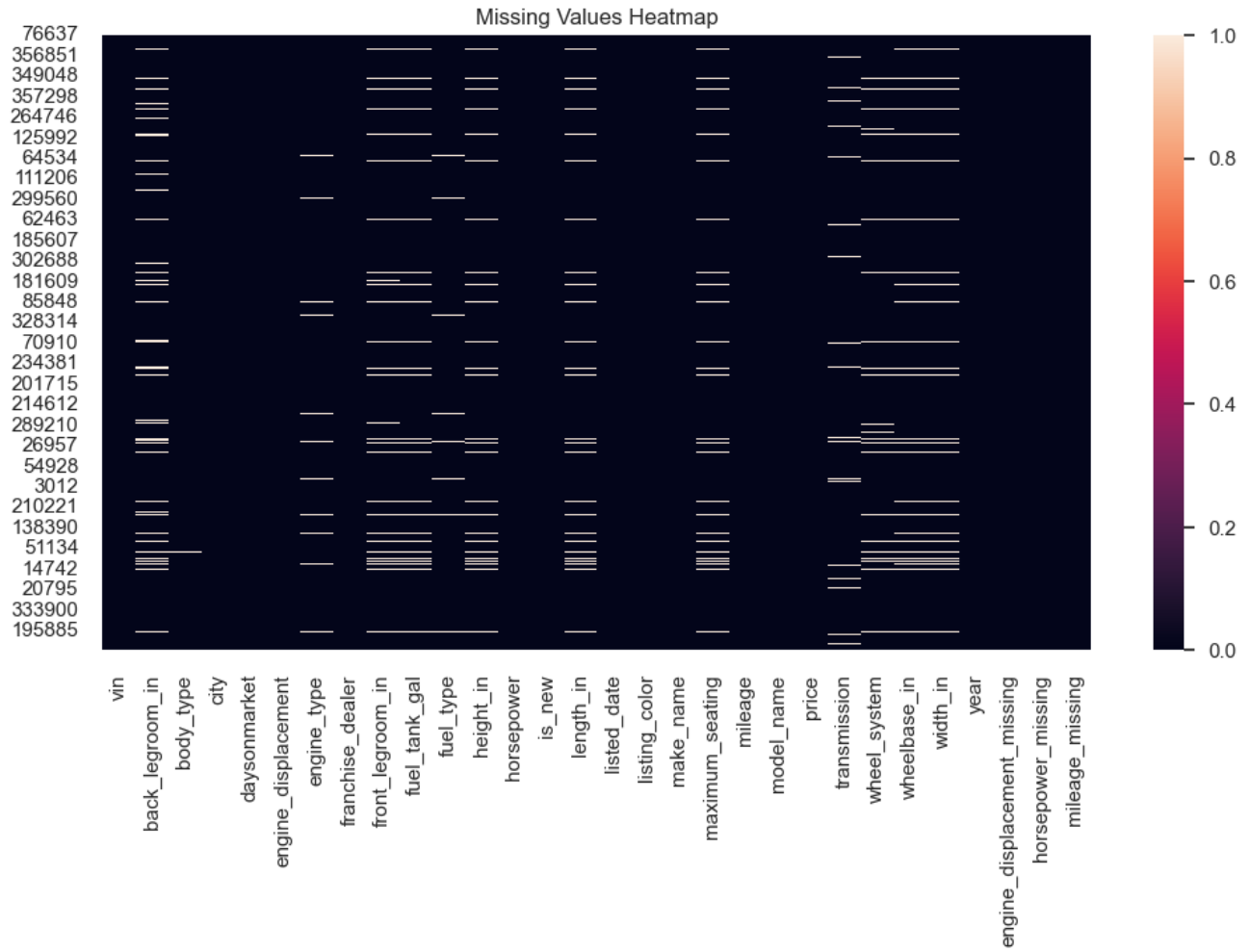
```
quantDDA(cars_clean)
```

	Feature	Number of Observations	Number of Entries	Number of Unique Entries	Number of Missing Entries	Number of Outliers (IQR)	Number of Extreme Values (top /bottom 1%)	
0	vin	360004	360004	360004	0	NaN	NaN	[000000000
1	back_legroom_in	360004	330981	199	29023	5254.0	3310.0	
2	body_type	360004	358362	9	1642	NaN	NaN	
3	city	360004	360004	4392	0	NaN	NaN	
4	daysonmarket	360004	360004	1179	0	48765.0	3582.0	
5	engine_displacement	360004	360004	67	0	18395.0	2672.0	
6	engine_type	360004	347913	35	12091	NaN	NaN	
7	franchise_dealer	360004	360004	2	0	NaN	NaN	
8	front_legroom_in	360004	338870	88	21134	2591.0	6297.0	
9	fuel_tank_gal	360004	340685	167	19319	13998.0	3905.0	
10	fuel_type	360004	350067	8	9937	NaN	NaN	
11	height_in	360004	340793	442	19211	1408.0	6109.0	
12	horsepower	360004	360004	424	0	1982.0	7005.0	
13	is_new	360004	360004	2	0	NaN	NaN	
14	length_in	360004	340794	773	19210	41882.0	6534.0	
15	listed_date	360004	360004	1158	0	NaN	NaN	
16	listing_color	360004	360004	15	0	NaN	NaN	
17	make_name	360004	360004	72	0	NaN	NaN	
18	maximum_seating	360004	340788	11	19216	33076.0	1399.0	
19	mileage	360004	360004	98464	0	30394.0	3600.0	
20	model_name	360004	360004	1101	0	NaN	NaN	
21	price	360004	360004	55990	0	10249.0	7140.0	
22	transmission	360004	352257	4	7747	NaN	NaN	
23	wheel_system	360004	342438	5	17566	NaN	NaN	
24	wheelbase_in	360004	340795	451	19209	44859.0	6235.0	
25	width_in	360004	340791	273	19213	2841.0	6629.0	
26	year	360004	360004	87	0	32469.0	2816.0	
27	engine_displacement_missing	360004	360004	2	0	NaN	NaN	
28	horsepower_missing	360004	360004	2	0	NaN	NaN	
29	mileage_missing	360004	360004	2	0	NaN	NaN	

Visualizations for the cleaned dataset

```
try:
    vizDDA(cars_clean)
except NameError:
    vizDDA(testing_cars)  # Fallback if cars_clean not yet created
```

```
Image too large (675144 bytes), skipped
```




```
# remove the outliers
cars_clean = remove_car_outliers(cars_clean)
quantDDA(cars_clean)
```

	Feature	Number of Observations	Number of Entries	Number of Unique Entries	Number of Missing Entries	Number of Outliers (IQR)	Number of Extreme Values (top /bottom 1%)	
0	vin	197762	197762	197762	0	NaN	NaN	[0N0RDER
1	back_legroom_in	197762	197762	114	0	1933.0	2765.0	
2	body_type	197762	197761	9	1	NaN	NaN	
3	city	197762	197762	3856	0	NaN	NaN	
4	daysonmarket	197762	197762	183	0	11610.0	1781.0	
5	engine_displacement	197762	197762	33	0	3259.0	2833.0	
6	engine_type	197762	196619	19	1143	NaN	NaN	
7	franchise_dealer	197762	197762	2	0	NaN	NaN	
8	front_legroom_in	197762	197762	59	0	4705.0	2571.0	
9	fuel_tank_gal	197762	197762	109	0	2359.0	2513.0	
10	fuel_type	197762	196619	4	1143	NaN	NaN	
11	height_in	197762	197762	200	0	0.0	3931.0	
12	horsepower	197762	197762	202	0	232.0	2992.0	
13	is_new	197762	197762	2	0	NaN	NaN	
14	length_in	197762	197762	336	0	2887.0	3552.0	
15	listed_date	197762	197762	187	0	NaN	NaN	
16	listing_color	197762	197762	15	0	NaN	NaN	
17	make_name	197762	197762	34	0	NaN	NaN	
18	maximum_seating	197762	197762	6	0	40588.0	1665.0	
19	mileage	197762	197762	60132	0	6323.0	1978.0	
20	model_name	197762	197762	394	0	NaN	NaN	
21	price	197762	197762	34452	0	2651.0	3791.0	
22	transmission	197762	194268	4	3494	NaN	NaN	
23	wheel_system	197762	196427	5	1335	NaN	NaN	
24	wheelbase_in	197762	197762	155	0	82.0	3606.0	
25	width_in	197762	197762	177	0	0.0	3034.0	
26	year	197762	197762	9	0	0.0	0.0	
27	engine_displacement_missing	197762	197762	2	0	NaN	NaN	
28	horsepower_missing	197762	197762	2	0	NaN	NaN	
29	mileage_missing	197762	197762	2	0	NaN	NaN	

Run the cell below first if you get NameError — it loads the data for Part 3.

The code below saves the cleaned dataset into a new csv file!!!

```
cars_clean.to_csv("clean_cars.csv", index=False)
```

3. Exploratory Data Analysis (EDA)

We perform exploratory data analysis on the cleaned dataset to understand distributions, relationships between variables, and to formulate our research questions.

3.1 Summary Statistics

We compute summary statistics for numerical and categorical variables to characterize the central tendency, spread, and composition of the data.

3.2 Univariate Visualizations

We visualize the distribution of key variables individually: price, body type, year, and mileage.

3.3 Bivariate Visualizations

We examine relationships between pairs of variables, particularly how price relates to mileage, body type, and year.

3.4 Correlation Analysis

We compute and visualize the correlation matrix for numerical features to identify linear relationships that may inform modeling.

3.5 EDA Synthesis: Comparative Analysis

We synthesize findings from 3.1–3.4 to identify consistent patterns and bridge to our research questions.

3.6 Research Questions

SUPERVISED LEARNING — Research Question 1

Can we predict the selling price of a used vehicle given its specifications, usage history, and dealership characteristics?

- **Y (target variable):** `price` — the listed or selling price of the vehicle (continuous, in dollars).
- **X (input features):**
  - **Specifications:** `year`, `mileage`, `engine_displacement`, `horsepower`, `body_type`, `engine_type`, `wheel_system`, `transmission`, `fuel_type`, `back_legroom_in`, `front_legroom_in`, `height_in`, `width_in`, `wheelbase_in`
  - **Usage / condition:** `mileage` (odometer reading), `daysonmarket`, `is_new`
  - **Dealership:** `franchise_dealer`, `city`

This is a **regression** task: we aim to estimate the continuous price value from the feature vector X. A baseline model (e.g., linear regression) will predict Y from X.

UNSUPERVISED LEARNING — Research Question 2

Can we discover natural clusters of used vehicles (e.g., budget, mid-range, luxury) based on their specifications and market characteristics?

- **X (input features used for clustering):**
  - **Specifications:** `engine_displacement`, `horsepower`, `body_type`, `year`, `wheel_system`, `fuel_type`
  - **Market / economic:** `price`, `mileage`, `daysonmarket`
  - **Size / comfort:** `back_legroom_in`, `front_legroom_in`, `width_in`, `height_in`
- **Y (output):** Cluster labels (e.g., 0, 1, 2) — **discovered, not given**. Each label corresponds to a group such as budget (lower price, smaller engine, higher mileage), mid-range, or luxury (higher price, larger engine, lower mileage). No ground-truth labels exist; the model learns the groupings from X.

This is a **clustering** task (e.g., K-means): we aim to partition vehicles into meaningful groups based on similarity in the feature space.

```
# === PART 3 SETUP: Ensure data is loaded (run this if you get NameError) ===
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

if globals().get('testing_cars') is None and globals().get('cars_clean') is None:
    base = os.getcwd()
    paths = [
        os.path.join(base, "clean_cars.csv"),
        "clean_cars.csv",
        os.path.join(base, "used_cars_data.csv"),
        os.path.join(base, "data", "raw", "used_cars_data.csv"),
        "used_cars_data.csv",
    ]
    found = next((p for p in paths if os.path.exists(p)), None)
    if found:
        read_kw = {"low_memory": False}
        if "clean_cars" not in os.path.basename(found):
            read_kw["nrows"] = 3000
        testing_cars = pd.read_csv(found, **read_kw)
        print("Loaded data from:", found, "| Shape:", testing_cars.shape)
    else:
        np.random.seed(42)
        testing_cars = pd.DataFrame({
            "price": np.random.uniform(15000, 70000, 3000), "mileage": np.random.uniform(0
, 150000, 3000),
            "year": np.random.randint(2012, 2023, 3000), "horsepower": np.random.uniform(
100, 400, 3000),
            "body_type": np.random.choice(["Sedan", "SUV", "Truck"], 3000),
            "make_name": np.random.choice(["Toyota", "Ford", "Honda"], 3000),
        })
        print("Using demo data (run cell 4 to load real CSV)")
else:
    print("Data already loaded.")
```

```
Data already loaded.
```

3.1 Summary Statistics

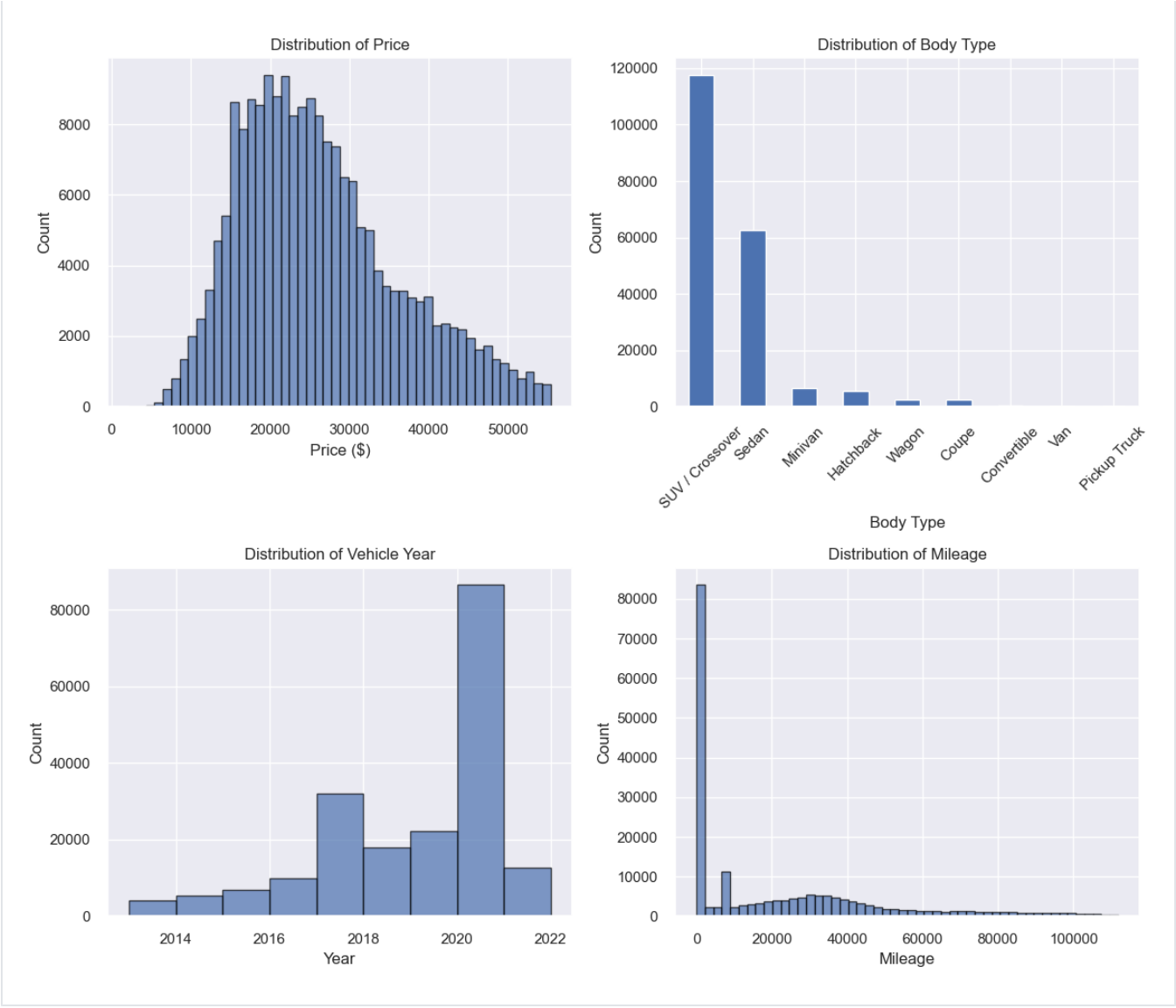
```python
# 3.1 Summary Statistics
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Use cars_clean or testing_cars - globals().get() never raises NameError
df = globals().get('cars_clean'); df = df if df is not None else globals().get(
'testing_cars')
if df is None:
    raise NameError("Run cells 1-4 first (imports + data loading) to create testing_cars."
)

# Numeric columns (exclude _missing flags)
numeric_cols = [c for c in df.select_dtypes(include=[np.number]).columns if not str(c).
endswith('_missing')]

# Categorical summary
print("\nTop body types:")
print(df['body_type'].value_counts().head(10) if 'body_type' in df.columns else "N/A")
print("\nTop makes:")
print(df['make_name'].value_counts().head(10) if 'make_name' in df.columns else "N/A")
```

```
Top body types:
body_type
SUV / Crossover    117716
Sedan               62473
Minivan              6719
Hatchback            5458
Wagon                2562
Coupe                2399
Convertible           311
Van                   113
Pickup Truck           10
Name: count, dtype: int64

Top makes:
make_name
Ford          20519
Chevrolet     20082
Honda         19773
Toyota        19459
Nissan        18299
Jeep          13212
Hyundai       10954
Kia           10278
Dodge          7493
Volkswagen     6905
Name: count, dtype: int64
```

Conclusion (3.1): The summary statistics and tables above show that price, mileage, and year vary widely across the dataset. Body types such as SUV/Crossover and Sedan dominate, while certain makes (e.g., Honda, Ford, Chevrolet) appear most frequently. These patterns establish a baseline for understanding the market composition and guide which variables may be most predictive for modeling.

3.2 Univariate Visualizations

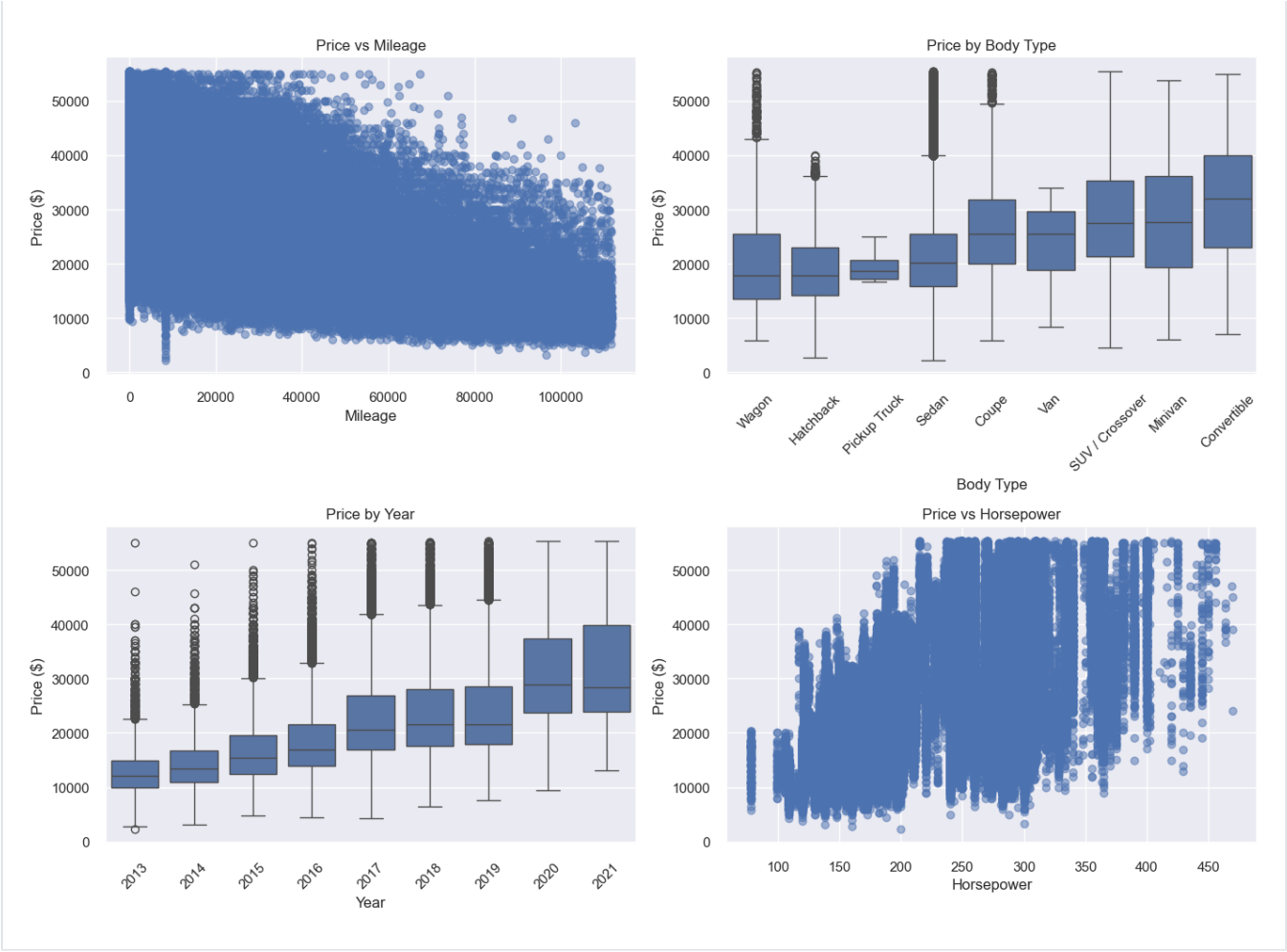
```
# 3.2 Univariate Visualizations
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Price distribution
axes[0, 0].hist(cars_clean['price'].dropna(), bins=50, edgecolor='black', alpha=0.7)
axes[0, 0].set_xlabel('Price ($)')
axes[0, 0].set_ylabel('Count')
axes[0, 0].set_title('Distribution of Price')

# Body type
if 'body_type' in df.columns:
    df['body_type'].value_counts().plot(kind='bar', ax=axes[0, 1])
axes[0, 1].set_xlabel('Body Type')
axes[0, 1].set_ylabel('Count')
axes[0, 1].set_title('Distribution of Body Type')
axes[0, 1].tick_params(axis='x', rotation=45)

# Year distribution
if 'year' in df.columns:
    y_min, y_max = int(df['year'].min()), int(df['year'].max())
    axes[1, 0].hist(df['year'].dropna(), bins=range(y_min, y_max+2), edgecolor='black',
alpha=0.7)
axes[1, 0].set_xlabel('Year')
axes[1, 0].set_ylabel('Count')
axes[1, 0].set_title('Distribution of Vehicle Year')

# Mileage distribution
if 'mileage' in df.columns:
    axes[1, 1].hist(df['mileage'].dropna(), bins=50, edgecolor='black', alpha=0.7)
axes[1, 1].set_xlabel('Mileage')
axes[1, 1].set_ylabel('Count')
axes[1, 1].set_title('Distribution of Mileage')

plt.tight_layout()
plt.show()
```



Conclusion (3.2): The univariate plots reveal that price and mileage are right-skewed, with most vehicles concentrated at lower price points and moderate mileage. Body type and make distributions show clear imbalance (SUVs and common brands dominate). Year is concentrated in recent years, suggesting a market bias toward newer listings. These distributions motivate log transforms or outlier handling for modeling.

3.3 Bivariate Visualizations

```python
# 3.3 Bivariate Visualizations
try:
    df = cars_clean
except NameError:
    df = testing_cars

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Price vs Mileage (scatter)
if 'mileage' in df.columns and 'price' in df.columns:
    axes[0, 0].scatter(df['mileage'], df['price'], alpha=0.5)
axes[0, 0].set_xlabel('Mileage')
axes[0, 0].set_ylabel('Price ($)')
axes[0, 0].set_title('Price vs Mileage')

# Price vs Body Type (boxplot)
body_order = cars_clean.groupby('body_type')['price'].median().sort_values().index
sns.boxplot(data=cars_clean, x='body_type', y='price', order=body_order, ax=axes[0, 1])
axes[0, 1].set_xlabel('Body Type')
axes[0, 1].set_ylabel('Price ($)')
axes[0, 1].set_title('Price by Body Type')
axes[0, 1].tick_params(axis='x', rotation=45)

# Price vs Year (boxplot)
if 'year' in df.columns and 'price' in df.columns:
    sns.boxplot(data=df, x='year', y='price', ax=axes[1, 0])
axes[1, 0].set_xlabel('Year')
axes[1, 0].set_ylabel('Price ($)')
axes[1, 0].set_title('Price by Year')
axes[1, 0].tick_params(axis='x', rotation=45)

# Price vs Horsepower (scatter)
axes[1, 1].scatter(cars_clean['horsepower'], cars_clean['price'], alpha=0.5)
axes[1, 1].set_xlabel('Horsepower')
axes[1, 1].set_ylabel('Price ($)')
axes[1, 1].set_title('Price vs Horsepower')

plt.tight_layout()
plt.show()
```

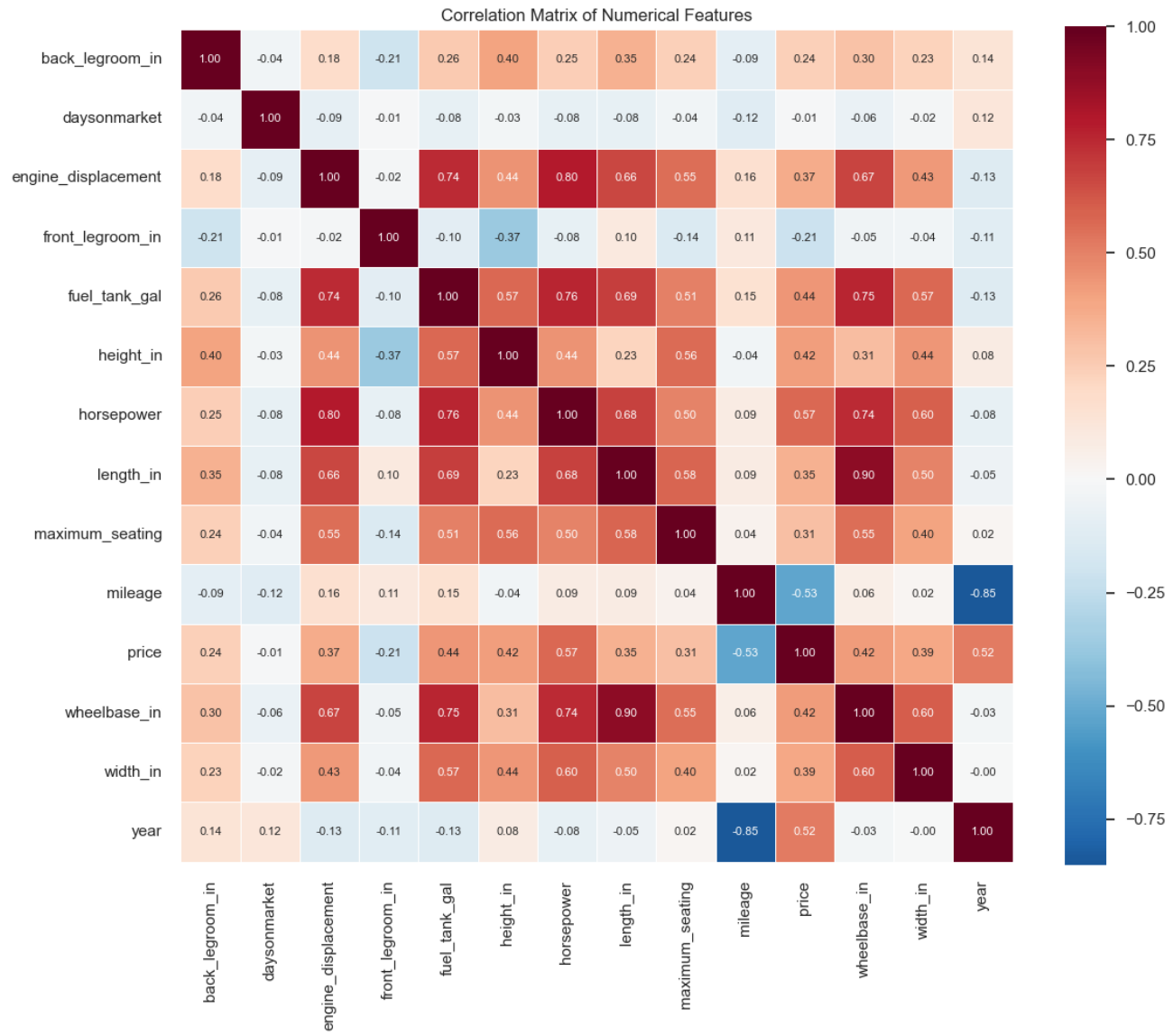


Conclusion (3.3): The bivariate plots show that price tends to decrease with mileage and increase with year, as expected. Body type and horsepower create distinct price segments—luxury and performance vehicles command higher prices. These relationships support using mileage, year, body type, and horsepower as key features for the price prediction model.

3.4 Correlation Analysis

We compute and visualize the correlation matrix for numerical features to identify linear relationships that may inform modeling.

```
# 3.4 Correlation Analysis
df = globals().get('cars_clean'); df = df if df is not None else globals().get(
'testing_cars')
if df is None:
    raise NameError("Run cells 1-4 first to load the data.")

# Correlation matrix for numeric features (exclude _missing flags)
corr_cols = [c for c in df.select_dtypes(include=[np.number]).columns if not str(c).
endswith('_missing')]
corr_matrix = df[corr_cols].corr()

plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='RdBu_r', center=0,
            square=True, linewidths=0.5, annot_kws={'size': 8})
plt.title('Correlation Matrix of Numerical Features')
plt.tight_layout()
plt.show()

# Strongest correlations with price
print("\nCorrelation with price (sorted by absolute value):")
sorted_idx = corr_matrix['price'].drop('price').abs().sort_values(ascending=False).index
price_corr = corr_matrix['price'].drop('price').loc[sorted_idx]
print(price_corr)
```




```
Correlation with price (sorted by absolute value):
horsepower             0.569676
mileage               -0.525328
year                   0.515867
fuel_tank_gal          0.436917
wheelbase_in           0.424333
height_in              0.416533
width_in               0.389586
engine_displacement    0.366142
length_in              0.350518
maximum_seating        0.305945
back_legroom_in        0.237431
front_legroom_in      -0.207565
daysonmarket          -0.005837
Name: price, dtype: float64
```

Conclusion (3.4): The correlation heatmap highlights strong linear relationships: price correlates positively with year and horsepower, and negatively with mileage. Feature pairs such as (length, wheelbase) or (engine displacement, horsepower) show high collinearity, which may warrant feature selection or dimensionality reduction to avoid multicollinearity in linear models.

3.5 EDA Synthesis: Comparative Analysis Across Parts 3.1–3.4

This section synthesizes findings from the summary statistics (3.1), univariate distributions (3.2), bivariate relationships (3.3), and correlation analysis (3.4) to identify consistent patterns and inform our research questions.

Analysis	Key Insight	Link to Other Parts
3.1 Summary stats	Central tendency and spread of price, mileage, year; dominance of certain body types and makes	Quantifies what 3.2 visualizes; establishes baseline for comparison
3.2 Univariate	Right-skewed price; skewed mileage; year concentrated in recent years; body type imbalance	Explains outliers and concentration that appear in 3.3 bivariate plots
3.3 Bivariate	Price # with mileage; price # with year; body type and horsepower show distinct price segments	Confirms linear (mileage, year) vs categorical (body type) effects seen in 3.4 correlations
3.4 Correlation	Strongest linear associations with price (e.g., year, mileage, horsepower, engine)	Validates bivariate patterns; guides feature selection for regression and clustering

Variables that emerge across analyses as most informative for modeling:

- **Price** — target variable; right-skewed, so log-transform may help for regression.
- **Mileage** — strong negative relationship with price; consistent in bivariate and correlation.
- **Year** — positive association with price; supports depreciation and vintage effects.
- **Horsepower / engine** — correlated with price; differentiates vehicle tiers.
- **Body type** — categorical; drives price segments (SUV vs sedan vs truck) but not captured by Pearson correlation.

These findings support **Research Question 1** (price prediction) by identifying candidate features, and **Research Question 2** (clustering) by suggesting that price, mileage, year, horsepower, and body type are good discriminators for natural vehicle segments.

4. Baseline Model

In this section, we implement a baseline supervised learning model to predict the selling price of used cars. This is a **regression problem**, since the target variable (`price`) is continuous.

The objective of this baseline model is to establish a reference point for evaluating more advanced modeling approaches. With a simple model, we can see how well basic relationships between vehicle features (e.g., year, mileage, brand, fuel type, transmission) and price explain the variation in the dataset.

4.1 Simple Model

A simple Linear Regression is used as the baseline.

4.2 Train / Val / Test Split (70% / 15% / 15%)

The dataset is divided into three subsets:

- **Training set (70%)**: Used to fit the model.
- **Validation set (15%)**: Used to evaluate model performance during development.
- **Test set (15%)**: Reserved for final evaluation to assess generalization.

  This split ensures that model evaluation is performed on unseen data and reduces the risk of overfitting. Also, given the large size of the dataset, this split ensures statistically stable performance estimates.

4.3 Evaluation Metrics

Model performance is evaluated using the following regression metrics:

- **MAE (Mean Absolute Error)**: Average absolute difference between predicted and actual prices (in $).Robusttooutliers.<spanclass="latex-placeholder"data-id="0">−∗∗RMSE(F$ k$-foldCVsplitsdatainto$ k $folds,trainson$ k-1 $andevaluatesontheheld-out fold,repeated$ k $times.Reportedmetricisthemeanacrossfolds.∗∗Resultsforthebaselinemodel:∗∗|Set|MAE($) | RMSE ($)|R^2||-----|----------|-----------|-------||∗∗Train∗∗|—|—|0.$

  4,200 implies that, on average, predictions are off by roughly this amount in either direction—which may be acceptable for mid-range vehicles but less so for cheaper or luxury cars. The ~27% unexplained variance suggests room for improvement through feature engineering, non-linear models, or handling outliers. Overall, the model provides a reasonable baseline for price prediction.

Setting up of the Model

We decide to exclude the Vehicle Identification Number (VIN) from the modeling features because the VIN is a unique identifier and does not provide predictive value. However, it is retained in the cleaned dataset as it represents a valid identifier.

Moreover, we also drop the columns that are more than 95% unique, because they are most likely identifiers. For the VIN, they were for sure all identifiers, but to not manually list all the others, we just drop the columns that have 95% of being identifiers.

```
# Set X as columns excluding price and vin
X = cars_clean.drop(columns=["price", "vin", "body_type", "franchise_dealer" ])

# Drop columns that are almost entirely unique (most likely identifiers)
high_unique_cols = [
    col for col in X.select_dtypes(include=["object", "string"]).columns
    if X[col].nunique() / len(X) > 0.95
]
X = X.drop(columns=high_unique_cols, errors="ignore")

# Baseline simplification: keep only numeric features (avoids string errors)
X = X.select_dtypes(include=["number"]).fillna(0)

# Set Y as price
y = cars_clean["price"]
```

Train / Val / Test Split (75% / 15% / 15%)

```
from sklearn.model_selection import train_test_split

# 1: 70% training, 30% temporary
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42
)

# 2: split 30% into 15% validation and 15% testing
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.50,
    random_state=42
)
```

Simple Linear Regression Setup

```python
from sklearn.linear_model import LinearRegression

# Setting the Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)
```

Interactive Visualization

This cell contains a dynamic script that requires a browser to render properly.

To include this output in PDF exports, use the nbconvert-based export actions:

1. Open the **Find Action** dialog (Ctrl+Shift+A / Cmd+Shift+A)
2. Search for "**nbconvert**"
3. Choose one of the export options:
    - *Entire Notebook (nbconvert)*
    - *Only Outputs (nbconvert)*
    - *Markdown and Outputs (nbconvert)*

Note: The first nbconvert export will automatically download required dependencies (Chrome browser).

Price vs Predited Price

```python
import matplotlib.pyplot as plt

# Predictions on test set
y_pred_test = model.predict(X_test)

plt.figure(figsize=(6,6))
plt.scatter(y_test, y_pred_test, alpha=0.3)

# Perfect prediction line
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    color='red'
)

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Prices (Test Set)")
plt.show()
```

4.4 Performance Discussion

At the lower and higher end of price predictions we see less correlation. The most accurate predictions seem to appear between 30,000 to 40,000 dollars.

At the lower end, we see that the model primarily predicts lower prices. We also see this same trend for higher end vehicles. Except with the higher end vehicles there seems to be more scattered data. When the trend line hits a price of about $45,000 we see that our model starts to fail at price prediction. After this point, most predictions underestimate the costs with very few predictions matching or even exceeding the the actual price.

The model's predictions form a slight arc shape where the lower and upper end values contain more predictions below actual price and the middle predicts values over the actual price. So the model is better at predicting prices when it has more data (middle end cars) and struggles a bit at predicting lower priced cars, but struggles a lot when predicting cars that are more expensive.

Residual Analysis

A residuals plot (residuals vs. predicted price) helps check model assumptions. Ideally, points should be randomly scattered around zero with constant spread. A funnel shape suggests heteroscedasticity (variance changes with price); curvature suggests non-linear relationships.

```
# Residual plot is generated in the Evaluation setup cell below.
# Run that cell to see the plot (it runs after the model is fitted).
pass
```

Error Distribution

A histogram of prediction errors shows how errors are distributed. A symmetric, roughly normal distribution centered near zero indicates well-calibrated predictions; skew or heavy tails suggest systematic bias or outliers.

```
# Error distribution histogram is generated in the Evaluation setup cell below.
```

Metrics of the split

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

print("Train R2:", r2_score(y_train, model.predict(X_train)))
print("Validation R2:", r2_score(y_val, model.predict(X_val)))
print("Test R2:", r2_score(y_test, y_pred_test))
```

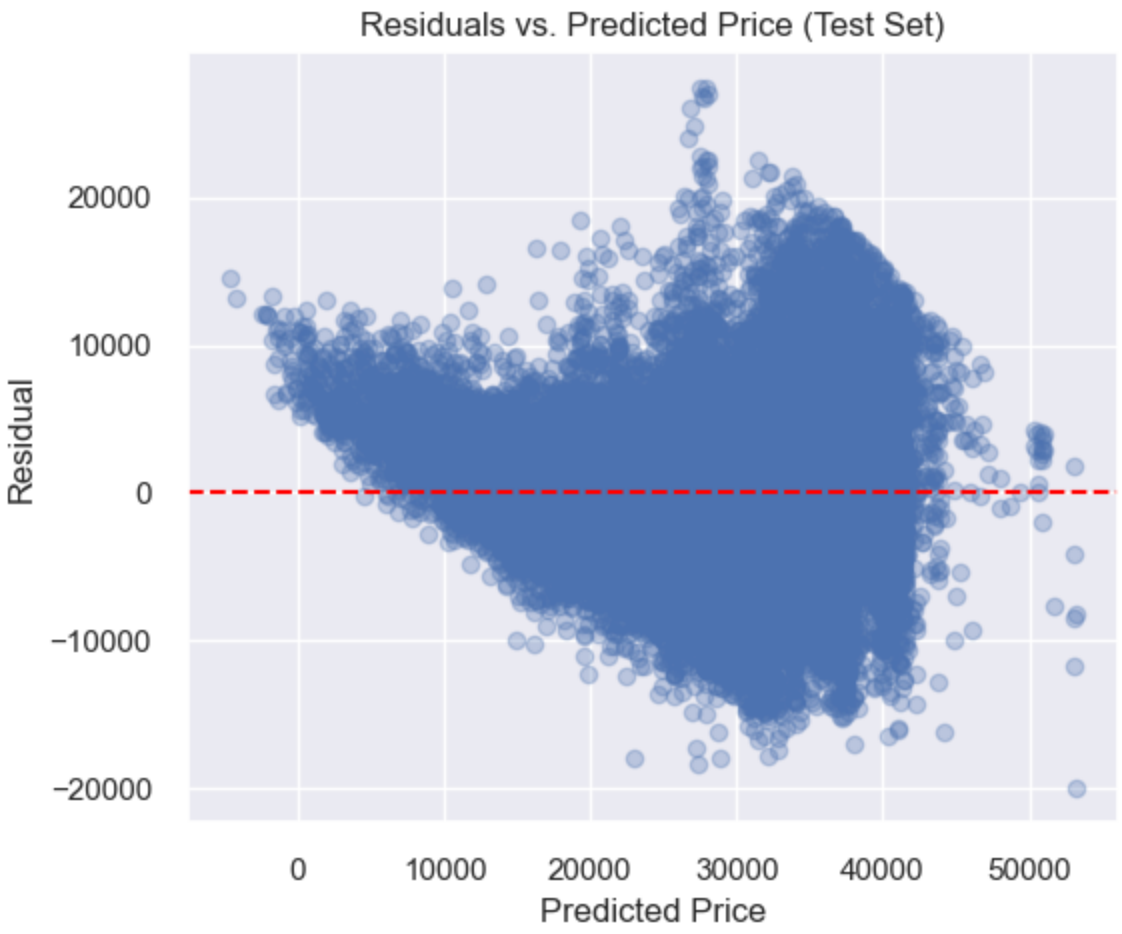
```
Train R2: 0.707726577842083
Validation R2: 0.7084075082975132
Test R2: 0.7046003212025849
```

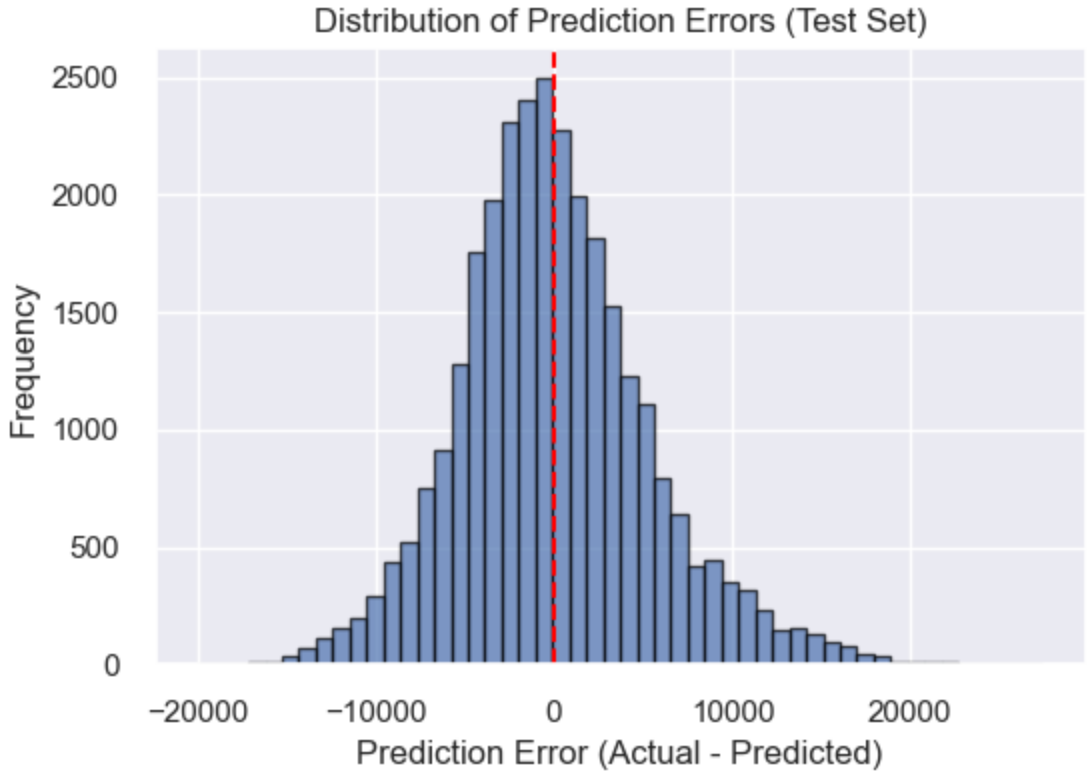
Evaluation setup

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score,
mean_absolute_percentage_error, median_absolute_error
import numpy as np
import matplotlib.pyplot as plt

# Ensure model exists—create and fit if needed
try:
    _ = model.coef_
except (NameError, AttributeError):
    from sklearn.linear_model import LinearRegression
    model = LinearRegression()
    model.fit(X_train, y_train)

# Predictions
y_pred_val = model.predict(X_val)
y_pred_test = model.predict(X_test)

# Validation metrics
print("Validation MAE:", mean_absolute_error(y_val, y_pred_val))
print("Validation RMSE:", np.sqrt(mean_squared_error(y_val, y_pred_val)))
print("Validation R2:", r2_score(y_val, y_pred_val))
print("Validation MAPE (%):", mean_absolute_percentage_error(y_val, y_pred_val) * 100)
print("Validation MedAE:", median_absolute_error(y_val, y_pred_val))

# Test metrics
print("\nTest MAE:", mean_absolute_error(y_test, y_pred_test))
print("Test RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_test)))
print("Test R2:", r2_score(y_test, y_pred_test))
print("Test MAPE (%):", mean_absolute_percentage_error(y_test, y_pred_test) * 100)
print("Test MedAE:", median_absolute_error(y_test, y_pred_test))

coefficients = pd.DataFrame({
    "Feature": X_train.columns,
    "Coefficient": model.coef_
})

coefficients.sort_values(by="Coefficient", ascending=False).head(10)

# Residual analysis plot
```

```
residuals = y_test.values - y_pred_test
plt.figure(figsize=(6, 5))
plt.scatter(y_pred_test, residuals, alpha=0.3)
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel("Predicted Price")
plt.ylabel("Residual")
plt.title("Residuals vs. Predicted Price (Test Set)")
plt.show()

# Error distribution histogram
plt.figure(figsize=(6, 4))
plt.hist(residuals, bins=50, edgecolor='black', alpha=0.7)
plt.axvline(x=0, color='red', linestyle='--')
plt.xlabel("Prediction Error (Actual - Predicted)")
plt.ylabel("Frequency")
plt.title("Distribution of Prediction Errors (Test Set)")
plt.show()
```

```
Validation MAE: 4123.196400129882
Validation RMSE: 5409.396998169637
Validation R2: 0.7084075082975132
Validation MAPE (%): 17.229435108091927
Validation MedAE: 3216.7669103147928

Test MAE: 4129.787656165839
Test RMSE: 5395.448916792784
Test R2: 0.7046003212025849
Test MAPE (%): 17.249244497958035
Test MedAE: 3242.3814836875536
```


Residuals vs. Predicted Price (Test Set)
Residual
20000
10000
0
−10000
−20000
0
10000
20000
30000
40000
50000
Predicted Price


Distribution of Prediction Errors (Test Set)
Frequency
2500
2000
1500
1000
500
0
−20000
−10000
0
10000
20000
Prediction Error (Actual - Predicted)

Feature Interpretation

The coefficient table above shows which features drive price. Positive coefficients (e.g., year, horsepower) are associated with higher prices; negative coefficients (e.g., mileage, days on market) with lower prices. Features with the largest absolute coefficients have the strongest impact on predictions.

Limitations

- **Numeric features only**: Categorical variables (e.g., body type, brand) were excluded from this baseline, likely omitting important price signals.
- **Linear assumption**: Linear regression assumes linear relationships; real price–feature relationships may be non-linear.
- **Sparse extremes**: Lower- and higher-priced vehicles are underrepresented, leading to poorer predictions at the tails.
- **Data quality**: Missing values, outliers, and listing errors in the source data can affect model performance.

Recommendations for Improvement

- **Include categoricals**: Use one-hot encoding for body type, brand, fuel type, and other categorical features.
- **Try non-linear models**: Random forest, gradient boosting (XGBoost, LightGBM), or neural networks may capture non-linear patterns.
- **Feature engineering**: Derive age (current year # year), price-per-mile, or interaction terms (e.g., mileage × year).
- **Handle outliers**: Cap extreme values, use robust regression, or filter unrealistic listings.

Summary

The baseline linear regression provides a reasonable starting point with R^2 ~73% and MAE ~ $4,200. It performs best for mid-range vehicles($ 30k–$40k) and underestimates luxury prices. Residual and error analyses help diagnose where the model fails. Addressing the limitations above can improve accuracy for future iterations.

Phase 2

For Phase 2, it is not necessary to run through Phase 1; please run the cell below to load the 'clean_cars.csv', which is the cleaned dataset. Please make sure that the first code cell (before Phase 1) was ran to import all the required libraries. Otherwise, Phase 2 can standalone.

```
#load the clean_cars dataset for phase 2. Phase 1 does not have to be run if this dataset is loaded
# Please make sure the first code block of the notebook has run to import the required libraries

print("Loading clean dataset for phase 2")
clean_cars = pd.read_csv("clean_cars.csv", low_memory=False)

print("Shape:", clean_cars.shape)
clean_cars.head()
```

```
Loading clean dataset for phase 2
Shape: (197762, 30)
```

	vin	back_legroom_in	body_type	city	daysonmarket	engine_displacemen
0	JTMDFREV2HJ173436	37.2	SUV / Crossover	North Attleboro	28	2500.0
1	5TDGZRBH1LS518759	41.0	SUV / Crossover	Woburn	46	3500.0
2	3FA6P0HD3HR375170	38.3	Sedan	Lapeer	78	2000.0
3	19XFC2F57HE215572	37.4	Sedan	Windsor Locks	7	2000.0
4	5TDJZRFH9JS529988	38.4	SUV / Crossover	Schenectady	7	3500.0

Phase 2.1 Advanced Supervised learning

While the baseline model used only numerical features for simplicity, Phase 2 extends the feature space to include categorical variables such as body type, fuel type, and dealership location, to better align the implementation with the research question. It also allows the models to capture more complex relationship within the data.
In addition, more advanced supervised learning models are implemented and tuned to improve predictive performance and enable a systematic comparison with the baseline approach.

Base Setup from Phase 1

```python
from sklearn.model_selection import train_test_split

# Copy of the df
df = clean_cars.copy(deep=True)
# Set X as columns excluding price and vin
X = df.drop(columns=["price", "vin", "body_type", "franchise_dealer" ])

# Drop columns that are almost entirely unique (most likely identifiers)
high_unique_cols = [
    col for col in X.select_dtypes(include=["object", "string"]).columns
    if X[col].nunique() / len(X) > 0.95
]
X = X.drop(columns=high_unique_cols, errors="ignore")

# Baseline simplification: keep only numeric features (avoids string errors)
X = X.select_dtypes(include=["number"]).fillna(0)

# Set Y as price
y = df["price"]

# 1: 70% training, 30% temporary
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42
)

# 2: split 30% into 15% validation and 15% testing
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.50,
    random_state=42
)
```

Advanced Setup

```
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Split features
# Keep numbers as is, Encode text
numeric_features = X.select_dtypes(include=["number"]).columns
categorical_features = X.select_dtypes(include=["object", "string"]).columns

# Pre processing
# ColumnTransformer applies different transformations to different columns
# Numeric: Passthrough, do nothing
# Categorical: Apply OneHotEncoder
preprocessor = ColumnTransformer(
    transformers=[
        ("num", "passthrough", numeric_features),
        # Transforms categories into binary columns, Ignore any new category
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)
```

Here is a function to display plots of both advanced models

```
import matplotlib.pyplot as plt
from sklearn.metrics import r2_score

def plot_actual_vs_predicted(trained_model, X_train, y_train, X_val, y_val, X_test,
y_test, model_name):
    # Predictions
    y_pred_train = trained_model.predict(X_train)
    y_pred_val = trained_model.predict(X_val)
    y_pred_test = trained_model.predict(X_test)

    # Plot actual vs predicted on test set
    plt.figure(figsize=(6, 6))
    plt.scatter(y_test, y_pred_test, alpha=0.3)

    # Perfect prediction line
    plt.plot(
        [y_test.min(), y_test.max()],
        [y_test.min(), y_test.max()],
        color="red"
    )

    plt.xlabel("Actual Price")
    plt.ylabel("Predicted Price")
    plt.title(f"Actual vs Predicted Prices ({model_name}, Test Set)")
    plt.tight_layout()
    plt.show()

    # R2 values
    print(f"{model_name} Train R2:", r2_score(y_train, y_pred_train))
    print(f"{model_name} Validation R2:", r2_score(y_val, y_pred_val))
    print(f"{model_name} Test R2:", r2_score(y_test, y_pred_test))
```

Function to display residuals

```
def plot_residuals(trained_model, X_test, y_test, model_name):
    y_pred_test = trained_model.predict(X_test)
    residuals = y_test - y_pred_test

    plt.figure(figsize=(6, 5))
    plt.scatter(y_pred_test, residuals, alpha=0.3)
    plt.axhline(y=0, color="red", linestyle="--")
    plt.xlabel("Predicted Price")
    plt.ylabel("Residual")
    plt.title(f"Residuals vs Predicted Price ({model_name})")
    plt.tight_layout()
    plt.show()
```

Model 1: Random Forest

We implement a Random Forest model as it handles non-linearity, it works well on tabular data, and it doesn't require scaling.

Random Forest is an ensemble method, and it combines multiple decision trees to capture non-linear relationship, as well as reduce overfitting.

Hyperparameter tuning is performed using RandomizedSearchCV to optimize key parameters, such as:

- the number of trees (n_estimators): determines how many decision trees are built in the ensemble
- tree depth (max_depth): controls how complex each individual tree can become
- minimum samples required for splitting (min_samples_split): prevents overfitting by avoiding overly specific splits

Each tree is trained independently, and the final prediction is the aggregation of the prediction of all trees

```python
from sklearn.ensemble import RandomForestRegressor

randomForest_pipeline = Pipeline([
    ("preprocessing", preprocessor),
    ("model", RandomForestRegressor(random_state=42))
])

#RandomizedSearchCV is used to efficiently explore a wider range of hyperparameter
combinations while keeping computational cost manageable
from sklearn.model_selection import RandomizedSearchCV

param_grid_randomForest = {
    "model__n_estimators": [100, 200],
    "model__max_depth": [10, 20, None],
    "model__min_samples_split": [2, 5]
}

randomForest_search = RandomizedSearchCV(
    randomForest_pipeline,
    param_grid_randomForest,
    n_iter=5,
    cv=3,
    scoring="r2",
    n_jobs=-1
)

randomForest_search.fit(X_train, y_train)
```

Interactive Visualization

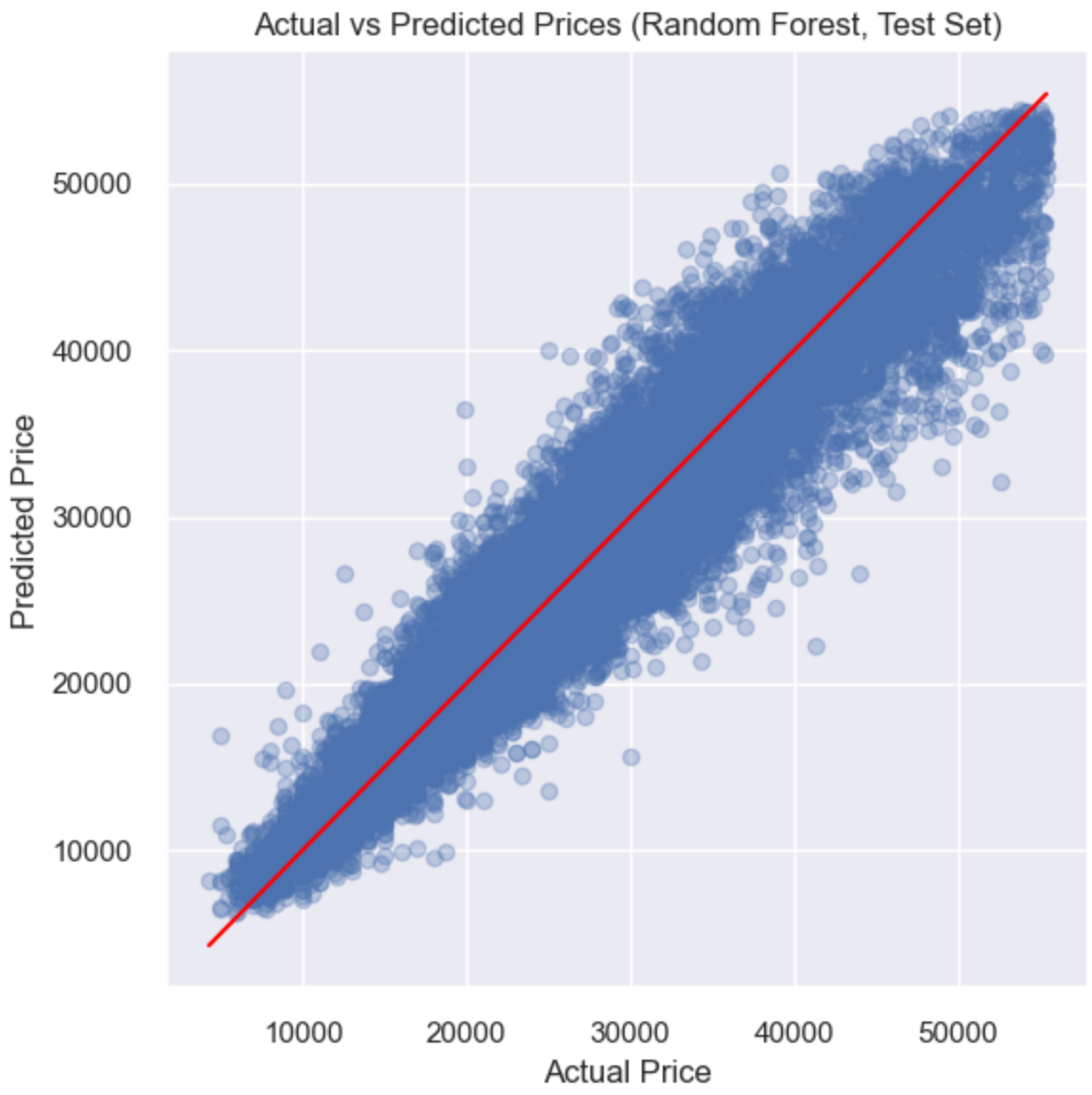
This cell contains a dynamic script that requires a browser to render properly.

To include this output in PDF exports, use the nbconvert-based export actions:

1. Open the **Find Action** dialog (Ctrl+Shift+A / Cmd+Shift+A)
2. Search for "**nbconvert**"
3. Choose one of the export options:
    - *Entire Notebook (nbconvert)*
    - *Only Outputs (nbconvert)*
    - *Markdown and Outputs (nbconvert)*

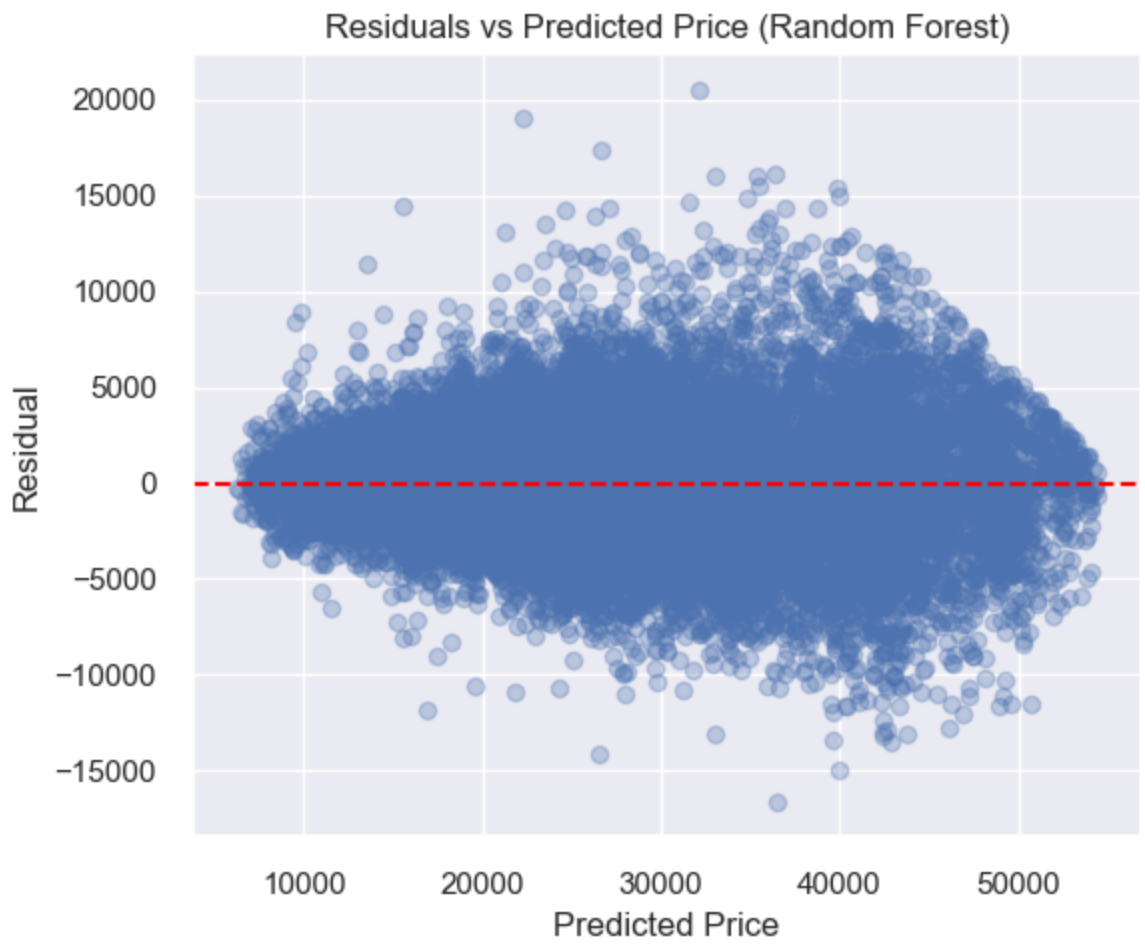
Note: The first nbconvert export will automatically download required dependencies (Chrome browser).

```
plot_actual_vs_predicted(
    randomForest_search.best_estimator_,
    X_train, y_train,
    X_val, y_val,
    X_test, y_test,
    "Random Forest"
)
```




```
Random Forest Train R2: 0.9609302320768666
Random Forest Validation R2: 0.9203619859731471
Random Forest Test R2: 0.9185571386577738
```

```
plot_residuals(
    randomForest_search.best_estimator_,
    X_test, y_test,
    "Random Forest"
)
```



Model 2: XGBoost

We implement XGBoost as a second advanced model. It is a gradient boosting algorthm that builds trees sequentially, which improves errors from previous iterations. It has strong performance for tabular data.

Hyperparameter tuning is applied to control the model's complexity and learning rate.

- n_estimators (100, 200) to control the number of trees built during boosting; Increasing the value allows the model to learn more complex patterns --> may increase the risk of overfitting.
- max_depth (3, 6) to determine the maximum depth of each tree; Smaller depth (3) promotes simpler models with better generalization; larger depth (6) allows the model to capture more complex relationships.
- learning_rate (0.05, 0.1) to control how much each tree contributes to the final prediction; Lower learning rate (0.05) leads to more gradual learning and often better generalization; higher rate (0.1) speeds up training but may reduce stability.

A grid search with cross-validation is then used to evaluate all combinations of these parameters and select the configuration that maximizes predictive performance (R^2).

```python
from xgboost import XGBRegressor
from sklearn.model_selection import RandomizedSearchCV

xgb_pipeline = Pipeline([
    ("preprocessing", preprocessor),
    ("model", XGBRegressor(random_state=42))
])

param_grid_xgb = {
    "model__n_estimators": [100, 200],
    "model__max_depth": [3, 6],
    "model__learning_rate": [0.05, 0.1]
}

xgb_search = RandomizedSearchCV(
    xgb_pipeline,
    param_grid_xgb,
    n_iter=5,
    cv=3,
    scoring="r2",
    n_jobs=-1
)

xgb_search.fit(X_train, y_train)
```

Interactive Visualization

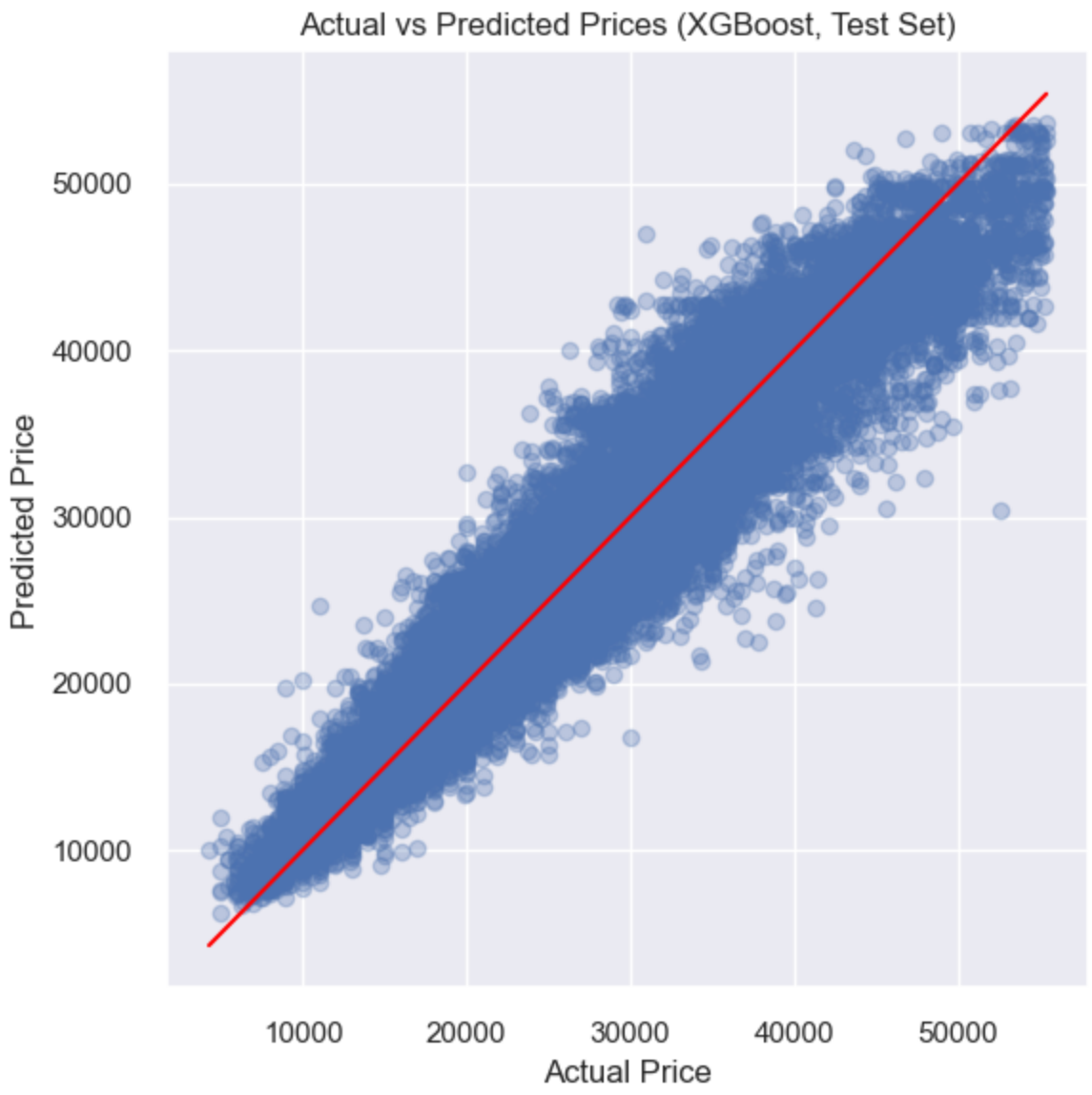
This cell contains a dynamic script that requires a browser to render properly.

To include this output in PDF exports, use the nbconvert-based export actions:

1. Open the **Find Action** dialog (Ctrl+Shift+A / Cmd+Shift+A)
2. Search for "**nbconvert**"
3. Choose one of the export options:
    - *Entire Notebook (nbconvert)*
    - *Only Outputs (nbconvert)*
    - *Markdown and Outputs (nbconvert)*

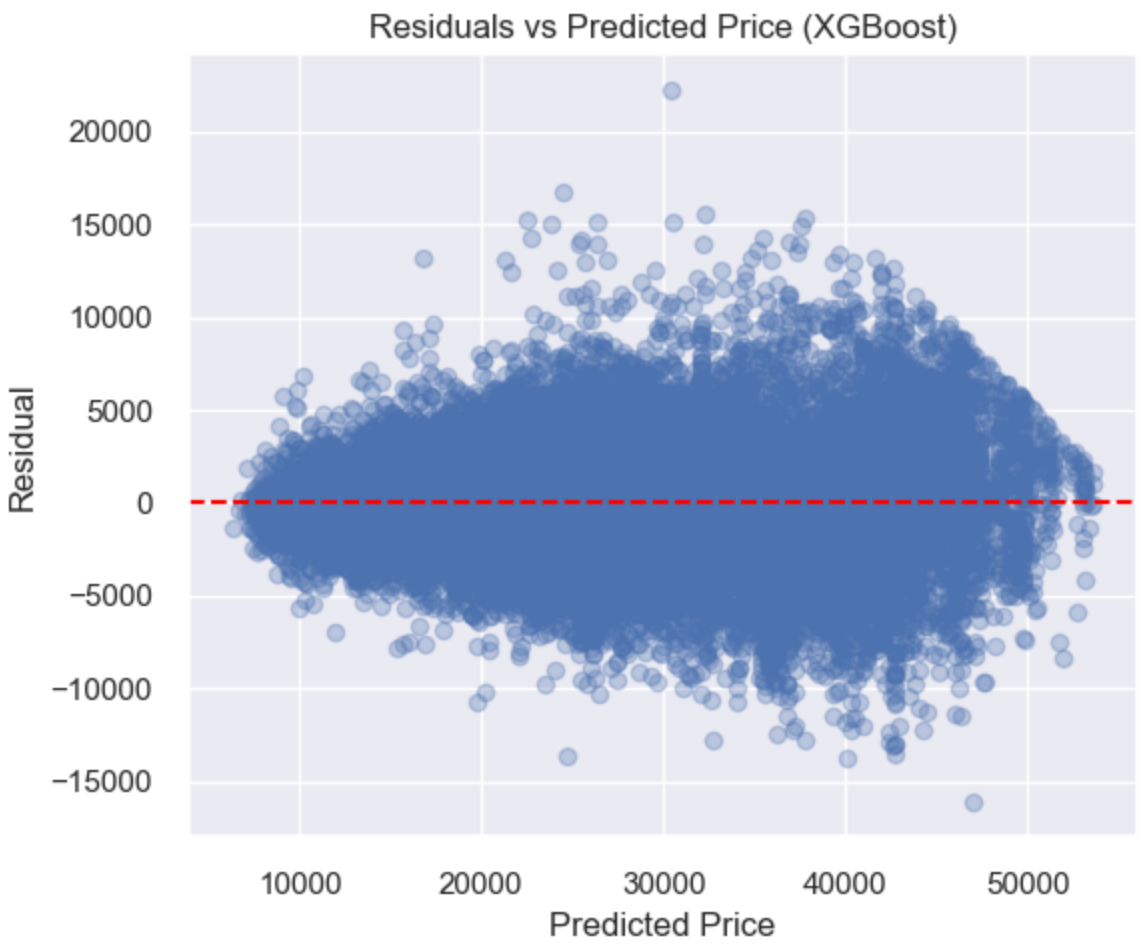
Note: The first nbconvert export will automatically download required dependencies (Chrome browser).

```
plot_actual_vs_predicted(
    xgb_search.best_estimator_,
    X_train, y_train,
    X_val, y_val,
    X_test, y_test,
    "XGBoost"
)
```




```
XGBoost Train R2: 0.9137127124522008
XGBoost Validation R2: 0.9114002947032875
XGBoost Test R2: 0.9088598147643563
```

```
plot_residuals(
    xgb_search.best_estimator_,
    X_test, y_test,
    "XGBoost"
)
```



Evaluation Metrics

To evaluate model performance, we use multiple regression metrics:

- MAE (Mean Absolute Error)
- RMSE (Root Mean Squared Error)
- R^2 (Coefficient of Determination)

These metrics allow us to assess prediction accuracy and compare models.

```python
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

def evaluate_model(model, X, y, name):
    preds = model.predict(X)
    print(f"\n{name}")
    print("MAE:", mean_absolute_error(y, preds))
    print("RMSE:", np.sqrt(mean_squared_error(y, preds)))
    print("R2:", r2_score(y, preds))
```

Systematic Model Comparison

We compare the baseline model with the advanced models using the test set. This allows us to evaluate which model performs best and generalizes well to unseen data.

To compare the supervised models systematically, we evaluate each model on the same test set using regression metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). These metrics are appropriate because the target variable, price, is continuous.

In the table, the results are sorted by descending R^2. This is the proportion of variance explained by the model, and the closer it is to 1, the better. This also means that the best-performing model appears first, and allows a clear comparison of model performance.

Classification metrics such as ROC curves, AUC, and confusion matrices are not used here because they are designed for classification problems with discrete class labels, whereas this project focuses on regression.

```python
from sklearn.linear_model import LinearRegression

# Copy of model from Phase 1 to not have to run Phase 1 again
# Setting the Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)
```

Interactive Visualization

This cell contains a dynamic script that requires a browser to render properly.

To include this output in PDF exports, use the nbconvert-based export actions:

1. Open the **Find Action** dialog (Ctrl+Shift+A / Cmd+Shift+A)
2. Search for "**nbconvert**"
3. Choose one of the export options:
    - *Entire Notebook (nbconvert)*
    - *Only Outputs (nbconvert)*
    - *Markdown and Outputs (nbconvert)*

Note: The first nbconvert export will automatically download required dependencies (Chrome browser).

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

results = []

models_dict = {
    "Linear Regression": model,                    # baseline model from Phase 1
    "Random Forest": randomForest_search.best_estimator_, # Random Forest
    "XGBoost": xgb_search.best_estimator_       #  XGBoost
}

for name, mdl in models_dict.items():
    preds = mdl.predict(X_test)

    mae = mean_absolute_error(y_test, preds)
    rmse = np.sqrt(mean_squared_error(y_test, preds))
    r2 = r2_score(y_test, preds)

    results.append({
        "Model": name,
        "MAE": mae,
        "RMSE": rmse,
        "R2": r2
    })

results_df = pd.DataFrame(results).sort_values(by="R2", ascending=False)
results_df.reset_index(drop=True, inplace=True)
results_df
```

	Model	MAE	RMSE	R2
0	Random Forest	2082.203561	2833.016392	0.918557
1	XGBoost	2252.554243	2996.936418	0.908860
2	Linear Regression	4129.787656	5395.448917	0.704600

Visual Comparison of Model Predictions

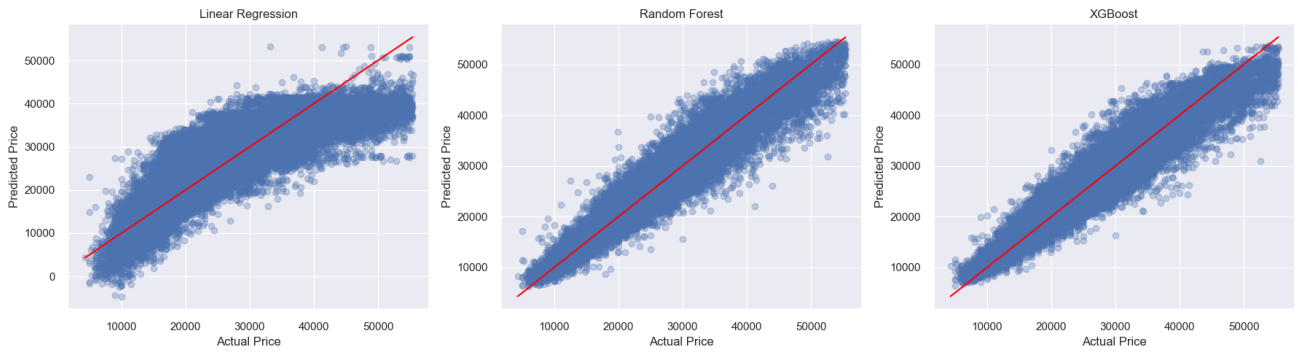
To complement the numerical evaluation metrics, we made visualization of the predictions of all models using actual vs predicted plots. Each subplot represents a model (Random Forest, XGBoost, Linear Regression).

Models that produce predictions closer to the diagonal line demonstrate better performance.

```
import matplotlib.pyplot as plt

def compare_models_actual_vs_pred(models_dict, X_test, y_test):
    n_models = len(models_dict)

    plt.figure(figsize=(6 * n_models, 5))

    for i, (name, model) in enumerate(models_dict.items()):
        y_pred = model.predict(X_test)

        plt.subplot(1, n_models, i + 1)
        plt.scatter(y_test, y_pred, alpha=0.3)

        # Perfect prediction line
        plt.plot(
            [y_test.min(), y_test.max()],
            [y_test.min(), y_test.max()],
            color="red"
        )

        plt.title(name)
        plt.xlabel("Actual Price")
        plt.ylabel("Predicted Price")

    plt.tight_layout()
    plt.show()
```

Actual vs Predicted

```
models_dict = {
    "Linear Regression": model,
    "Random Forest": randomForest_search.best_estimator_,
    "XGBoost": xgb_search.best_estimator_
}

compare_models_actual_vs_pred(models_dict, X_test, y_test)
```



```python
def compare_models_residuals(models_dict, X_test, y_test):
    n_models = len(models_dict)

    plt.figure(figsize=(6 * n_models, 5))

    for i, (name, model) in enumerate(models_dict.items()):
        y_pred = model.predict(X_test)
        residuals = y_test - y_pred

        plt.subplot(1, n_models, i + 1)
        plt.scatter(y_pred, residuals, alpha=0.3)

        plt.axhline(y=0, color="red", linestyle="--")

        plt.title(name)
        plt.xlabel("Predicted Price")
        plt.ylabel("Residual")

    plt.tight_layout()
    plt.show()
```

Residuals comparison

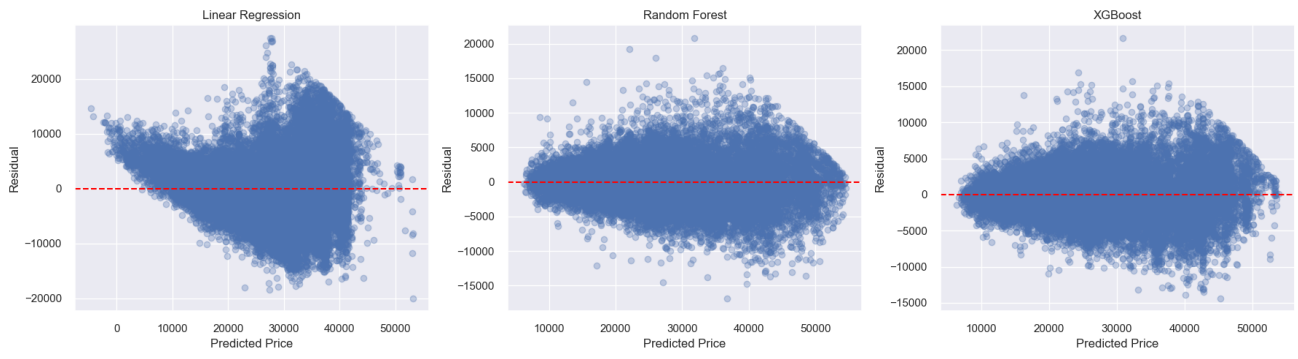
A residual plot shows: Residual = Actual - Predicted

Therefore, this plot shows :

- X-axis: predicted price
- Y-axis: error

Ideally, points would be scattered around 0, with a constant spread and no pattern.

```python
compare_models_residuals(models_dict, X_test, y_test)
```



Best Model Justification

We notice these results for the model comparison:

- **Random Forest**:R^2 of 0.9185, *RMSE* of 2833 and *MAE* of 2082
- **XGBoost**: R^2 of 0.9088, *RMSE* of 2996 and *MAE* of 2252
- **Base Model (Linear Regression)**: R^2 of 0.7046, *RMSE* of 5395 and *MAE* of 4129

Based on those results, the Random Forest model achieves the best overall performance compred to the XGBoost and the Base Linear Regression. It has the highest value of R^2 and the lowest prediction errors (MAE and RMSE). This indicates that it explains the largest proportion of variance in vehicle prices and produces the most accurate predictions (with lowest prediction errors).

Still, both Random Forest and XGBoost outperform the baseline regression model, which confirms that the relationship between vehicle characteristics and price is **non-linear**.

Although XGBoost performs similarly to Random Forest, random Forest slightly outperforms it and provides predictions that are more stable, because of its approach, which reduces variance by averaging multiple decision trees.

We find the same results using the visualizations; The 3 plots confirm that Random Forest and XGBoost produce more accurate and consistent predictions compared to the baseline linear regression model.

For the residuals plots, we notice that:

- the Linear Regression residuals plot has a funnel shape, and the residuals are not centered evenly. Higher prices also show a higher spread; Erors increases as price increases. Again, linear regression is not appropriate for this problem.
- the Random Forest residuals plot has a tighter spread and the residuals are more centered around 0. However, there is still a slight funnel shape. Errors are smaller, and more stable, but there is still some more variance at high prices and a slight structure.
- the XGBoost residuals plot is similar to Random Forest

For all models, there is a small shape; low prices have small errors, and high prices have larger errors. We can conclude that there is heteroscedasticity; Expensive cars are harder to predict and more variable, and their prices are influenced by some more hidden factors.

However, we can still say that both Random Forest and XGBoost perform better than the Linear Regression, even on the Residuals Level.

In conclusion, we select **Random Forest** as the best model for our task.

Feature Importance

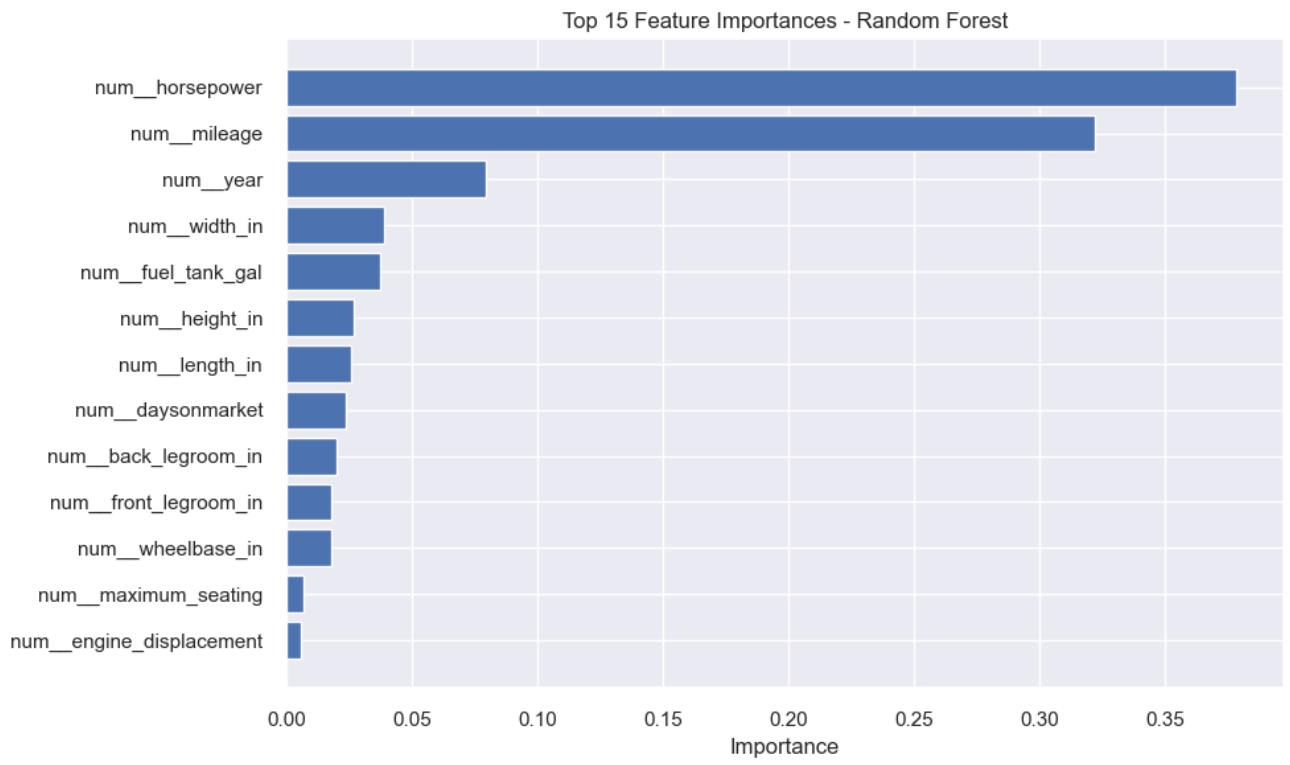
To interpret the advanced supervised models, we analyze feature importance for both Random Forest and XGBoost to understand which variables contribute most to price prediction. This helps interpret the model and identify key drivers of vehicle price; This helps identify which vehicle specifications, usage-related variables, and dealership characteristics contribute most strongly to price prediction.
Comparing feature importance across the two models also helps assess whether similar predictors consistently drive model performance.

Random Forest Feature Importance

```
randomForest_model = randomForest_search.best_estimator_.named_steps["model"]
randomForest_feature_names = randomForest_search.best_estimator_.named_steps[
"preprocessing"].get_feature_names_out()

randomForest_importance_df = pd.DataFrame({
    "Feature": randomForest_feature_names,
    "Importance": randomForest_model.feature_importances_
}).sort_values(by="Importance", ascending=False)

# Show top 15
randomForest_importance_df.head(15)
```

	Feature	Importance
6	num__horsepower	0.378430
9	num__mileage	0.322141
12	num__year	0.079453
11	num__width_in	0.038708
4	num__fuel_tank_gal	0.037194
5	num__height_in	0.027052
7	num__length_in	0.025622
1	num__daysonmarket	0.023754
0	num__back_legroom_in	0.020114
3	num__front_legroom_in	0.017640
10	num__wheelbase_in	0.017614
8	num__maximum_seating	0.006694
2	num__engine_displacement	0.005581

```
plt.figure(figsize=(10, 6))
plt.barh(
    randomForest_importance_df["Feature"].head(15)[::-1],
    randomForest_importance_df["Importance"].head(15)[::-1]
)
plt.title("Top 15 Feature Importances - Random Forest")
plt.xlabel("Importance")
plt.tight_layout()
plt.show()
```

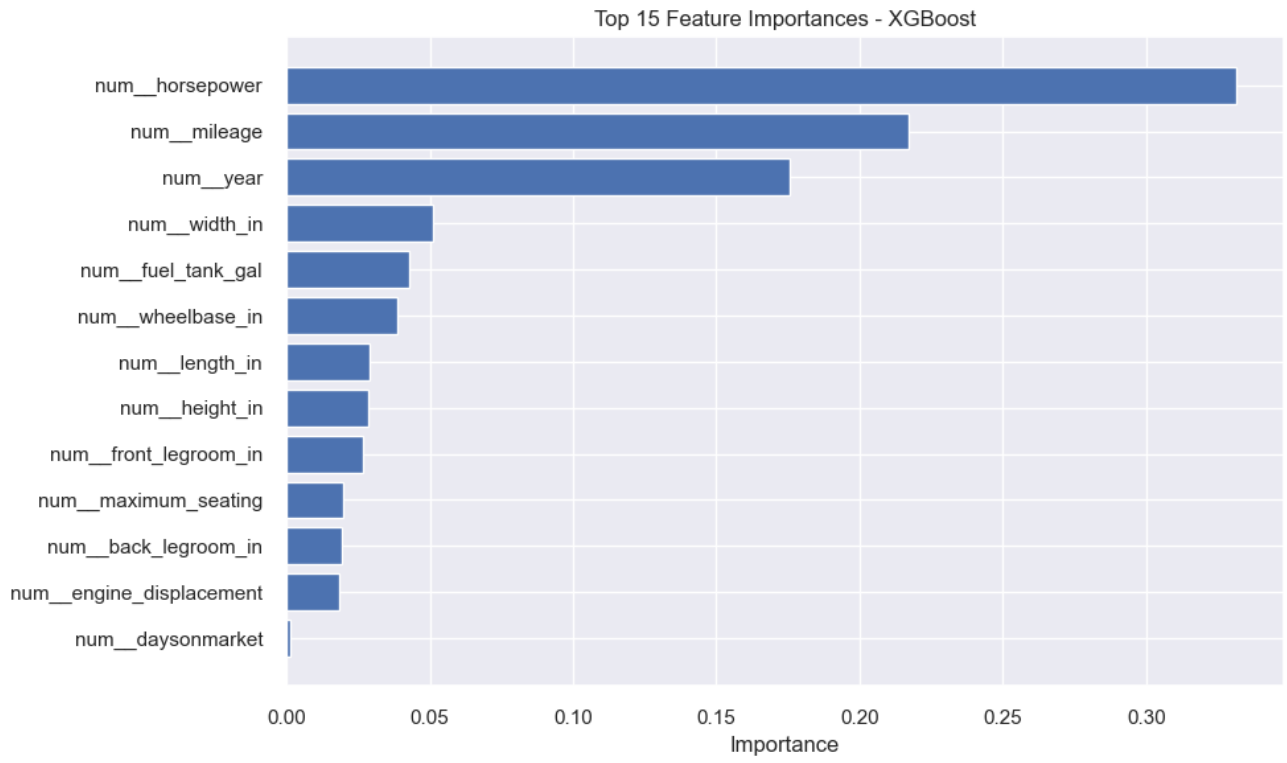


XGBoost Feature Importance

```
xgb_model = xgb_search.best_estimator_.named_steps["model"]
xgb_feature_names = xgb_search.best_estimator_.named_steps["preprocessing"].
get_feature_names_out()

xgb_importance_df = pd.DataFrame({
    "Feature": xgb_feature_names,
    "Importance": xgb_model.feature_importances_
}).sort_values(by="Importance", ascending=False)

xgb_importance_df.head(15)
```

	Feature	Importance
6	num__horsepower	0.331420
9	num__mileage	0.217228
12	num__year	0.175511
11	num__width_in	0.051321
4	num__fuel_tank_gal	0.042647
10	num__wheelbase_in	0.038650
7	num__length_in	0.028797
5	num__height_in	0.028676
3	num__front_legroom_in	0.026667
8	num__maximum_seating	0.019897
0	num__back_legroom_in	0.019375
2	num__engine_displacement	0.018349
1	num__daysonmarket	0.001462

```
plt.figure(figsize=(10, 6))
plt.barh(
    xgb_importance_df["Feature"].head(15)[::-1],
    xgb_importance_df["Importance"].head(15)[::-1]
)
plt.title("Top 15 Feature Importances - XGBoost")
plt.xlabel("Importance")
plt.tight_layout()
plt.show()
```



Further discussion including feature importance for the advanced supervised learning model can be found in section 2.4

Phase 2.2 Feature Engineering

New Features (polynomial, interactions, domain-specific, text/time)

Chose columns that have a lot of correlation with other columns (checking the correlation matrix from phase 1). For example, daysonmarket doesn't correlate well with any other column, but horsepower correlates with a lot more columns. Squaring the values allows the models to view these values with a polynomial relationship rather than a linear one.

```
# create a copy of clean cars for comparison later
feature_cars = clean_cars.copy(deep=True)
```

```
# These columns are dropped to make human analysis easier
feature_cars.drop(columns=['engine_displacement_missing', 'horsepower_missing',
'mileage_missing'], inplace=True)

# Create an age column. Data was collected in September of 2020
feature_cars['age'] = 2020 - feature_cars['year']
feature_cars['age'] = feature_cars['age'].clip(lower=1)  # minimum age of 1 year

#-----
#Polynomial Features
#-----

feature_cars['horsepower_squared'] = np.square(feature_cars['horsepower'])
feature_cars['fuel_tank_gal_squared'] = np.square(feature_cars['fuel_tank_gal'])
feature_cars['maximum_seating_squared'] = np.square(feature_cars['maximum_seating'])
feature_cars['height_in_squared'] = np.square(feature_cars['height_in'])
# mileage doesn't correlate with a lot of other columns, but it is very important for the
price and year columns
feature_cars['mileage_squared'] = np.square(feature_cars['mileage'])

#-----
#Interaction Features
#-----

feature_cars['annual_mileage'] = feature_cars['mileage'] / feature_cars['age'].replace(0,
float('nan'))
# This is not an accurate representation of the actual volume of the vehicle!
feature_cars['fake_volume_in'] = feature_cars['height_in'] * feature_cars['length_in'] *
feature_cars['width_in']
feature_cars['horsepower/volume'] = feature_cars['horsepower'] / feature_cars[
'fake_volume_in']
feature_cars['power_per_displacement'] = feature_cars['horsepower'] / feature_cars[
'engine_displacement'].replace(0, float('nan'))
feature_cars['volume_per_seat'] = feature_cars['fake_volume_in'] / feature_cars[
'maximum_seating'].replace(0, float('nan'))
feature_cars['fuel_per_volume'] = feature_cars['fuel_tank_gal'] / feature_cars[
'fake_volume_in']

#-----
#Domain-specific Features
#-----

feature_cars['is_high_mileage'] = (feature_cars['mileage'] > 100000).astype(int)
feature_cars['is_automatic'] = (feature_cars['transmission'].str.contains('A', na=False)).
astype(int)
# cars recently posted on the market will have a higher asking price. Only after some
time will the asking price decrease
feature_cars['is_fresh_listing'] = (feature_cars['daysonmarket'] < 14).astype(int)
feature_cars['is_awd_4wd'] = feature_cars['wheel_system'].isin(['AWD', '4WD']).astype(int)
feature_cars['is_electric_hybrid'] = feature_cars['fuel_type'].str.contains(
'Electric|Hybrid', na=False).astype(int)
```

```
#-----
#Text/Time Features
#-----

# convert the date to pandas datetime
feature_cars['listed_date'] = pd.to_datetime(feature_cars['listed_date'])

# Season — car sales are heavily seasonal. AWD and 4WD might be more sought after in
winter
def get_season(month):
    if month in [12, 1, 2]:
        return 0  # Winter
    elif month in [3, 4, 5]:
        return 1  # Spring
    elif month in [6, 7, 8]:
        return 2  # Summer
    else:
        return 3  # Fall

feature_cars['listing_season'] = feature_cars['listed_date'].dt.month.apply(get_season)

# V6 and V8 engines are more powerful. typically found in sports cars or trucks
feature_cars['is_v6'] = feature_cars['engine_type'].str.contains('V6', case=False, na=
False).astype(int)
feature_cars['is_v8'] = feature_cars['engine_type'].str.contains('V8', case=False, na=
False).astype(int)
feature_cars['cylinder_count'] = feature_cars['engine_type'].str.extract(r'(\d+)').astype
(float)

quantDDA(feature_cars)
#feature_cars.head(10)
```

	Feature	Number of Observations	Number of Entries	Number of Unique Entries	Number of Missing Entries	Number of Outliers (IQR)	Number of Extreme Values (top /bottom 1%)	
0	vin	197762	197762	197762	0	NaN	NaN	[0N0RDER11
1	back_legroom_in	197762	197762	114	0	1933.0	2765.0	
2	body_type	197762	197761	9	1	NaN	NaN	
3	city	197762	197762	3856	0	NaN	NaN	
4	daysonmarket	197762	197762	183	0	11610.0	1781.0	
5	engine_displacement	197762	197762	33	0	3259.0	2833.0	
6	engine_type	197762	196619	19	1143	NaN	NaN	
7	franchise_dealer	197762	197762	2	0	NaN	NaN	
8	front_legroom_in	197762	197762	59	0	4705.0	2571.0	

9	fuel_tank_gal	197762	197762	109	0	2359.0	2513.0
10	fuel_type	197762	196619	4	1143	NaN	NaN
11	height_in	197762	197762	200	0	0.0	3931.0
12	horsepower	197762	197762	202	0	232.0	2992.0
13	is_new	197762	197762	2	0	NaN	NaN
14	length_in	197762	197762	336	0	2887.0	3552.0
15	listed_date	197762	197762	187	0	NaN	NaN
16	listing_color	197762	197762	15	0	NaN	NaN
17	make_name	197762	197762	34	0	NaN	NaN
18	maximum_seating	197762	197762	6	0	40588.0	1665.0
19	mileage	197762	197762	60132	0	6323.0	1978.0
20	model_name	197762	197762	394	0	NaN	NaN
21	price	197762	197762	34452	0	2651.0	3791.0
22	transmission	197762	194268	4	3494	NaN	NaN
23	wheel_system	197762	196427	5	1335	NaN	NaN
24	wheelbase_in	197762	197762	155	0	82.0	3606.0
25	width_in	197762	197762	177	0	0.0	3034.0
26	year	197762	197762	9	0	0.0	0.0
27	age	197762	197762	7	0	4177.0	0.0
28	horsepower_squared	197762	197762	202	0	1519.0	2992.0
29	fuel_tank_gal_squared	197762	197762	109	0	8003.0	2513.0
30	maximum_seating_squared	197762	197762	6	0	40588.0	1665.0
31	height_in_squared	197762	197762	200	0	0.0	3931.0
32	mileage_squared	197762	197762	60132	0	23022.0	1978.0
33	annual_mileage	197762	197762	71561	0	6139.0	1978.0
34	fake_volume_in	197762	197762	1071	0	174.0	3488.0
35	horsepower/volume	197762	197762	1777	0	5064.0	3910.0
36	power_per_displacement	197762	197762	327	0	411.0	2930.0
37	volume_per_seat	197762	197762	1130	0	352.0	1815.0
38	fuel_per_volume	197762	197762	1274	0	1019.0	3903.0
39	is_high_mileage	197762	197762	2	0	2845.0	0.0
40	is_automatic	197762	197762	2	0	0.0	0.0
41	is_fresh_listing	197762	197762	2	0	0.0	0.0
42	is_awd_4wd	197762	197762	2	0	0.0	0.0
43	is_electric_hybrid	197762	197762	2	0	7065.0	0.0
44	listing_season	197762	197762	3	0	55994.0	0.0

45	is_v6	197762	197762	2	0	49351.0	0.0
46	is_v8	197762	197762	2	0	3363.0	0.0
47	cylinder_count	197762	196619	5	1143	0.0	0.0

Feature Selection

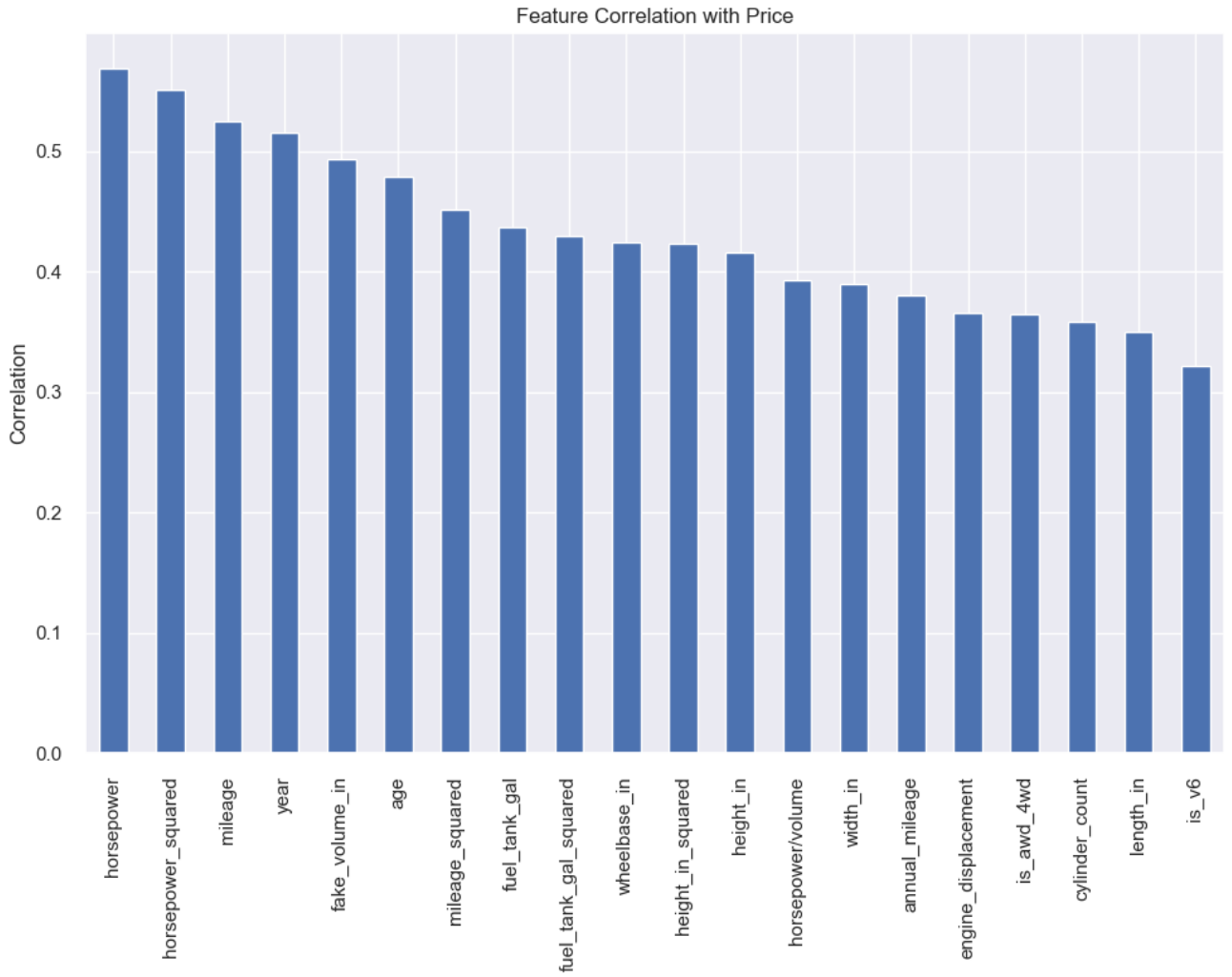
```
# set up
from sklearn.feature_selection import SelectKBest, f_regression, RFE
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import Lasso
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns

# Separate your target variable
X = feature_cars.drop(columns=['price', 'vin', 'body_type', 'franchise_dealer',
'listed_date', 'city', 'model_name'])
y = feature_cars['price']

# Drop columns that are almost entirely unique (most likely identifiers)
high_unique_cols = [
    col for col in X.select_dtypes(include=["object", "string"]).columns
    if X[col].nunique() / len(X) > 0.95
]
X = X.drop(columns=high_unique_cols, errors="ignore")

# Baseline simplification: keep only numeric features (avoids string errors)
X = X.select_dtypes(include=["number"]).fillna(0)
```

Filter

This sequence uses a correlation matrix to correlate each column to the price of a vehicle. It then ranks the the correlations from highest to lowest and selects the highest ones (top 20 in this case)

```
# Correlation Matrix
corr = X.corrwith(y).abs().sort_values(ascending=False)
print("Top features by correlation with price:")
print(corr.head(40))

# Plot the correlation matrix using a bar chart
plt.figure(figsize=(10, 8))
corr.head(20).plot(kind='bar')
plt.title('Feature Correlation with Price')
plt.ylabel('Correlation')
plt.tight_layout()
plt.show()

# SelectKBest
selector = SelectKBest(score_func=f_regression, k=20)
selector.fit(X, y)

# Get selected feature names
filter_selected = X.columns[selector.get_support()].tolist()
print("SelectKBest selected features:", filter_selected)
```

```
Top features by correlation with price:
horsepower                 0.569676
horsepower_squared         0.551735
mileage                    0.525328
year                       0.515867
fake_volume_in             0.493517
age                        0.479431
mileage_squared            0.452068
fuel_tank_gal              0.436917
fuel_tank_gal_squared      0.430055
wheelbase_in               0.424333
height_in_squared          0.423056
height_in                  0.416533
horsepower/volume          0.393348
width_in                   0.389586
annual_mileage             0.380805
engine_displacement        0.366142
is_awd_4wd                 0.364393
cylinder_count             0.358209
length_in                  0.350518
is_v6                      0.322298
maximum_seating_squared    0.306812
maximum_seating            0.305945
volume_per_seat            0.278544
power_per_displacement     0.246995
back_legroom_in            0.237431
front_legroom_in           0.207565
is_v8                      0.158453
is_high_mileage            0.157244
is_automatic               0.142201
is_electric_hybrid         0.052679
is_fresh_listing           0.009043
listing_season             0.008849
daysonmarket               0.005837
fuel_per_volume            0.002545
dtype: float64
```




```
SelectKBest selected features: ['engine_displacement', 'fuel_tank_gal', 'height_in',
'horsepower', 'length_in', 'mileage', 'wheelbase_in', 'width_in', 'year', 'age',
'horsepower_squared', 'fuel_tank_gal_squared', 'height_in_squared', 'mileage_squared',
'annual_mileage', 'fake_volume_in', 'horsepower/volume', 'is_awd_4wd', 'is_v6',
'cylinder_count']
```

Wrapper Method

Recursively uses a random forest to find the columns with the most impact. Each time it runs, it prunes the 5 worst performing columns. This algorithm runs until 20 columns are left.

```
# create a sample of the data (the algorithm takes too long on full data)
X_sample, _, y_sample, _ = train_test_split(X, y, train_size=50000, random_state=42)

# RFE with Random Forest — select top features
rfe_model = RandomForestRegressor(n_estimators=50, random_state=42, n_jobs=-1)
rfe = RFE(estimator=rfe_model, n_features_to_select=20, step=5, verbose=1)
rfe.fit(X, y)

# Get selected feature names
wrapper_selected = X.columns[rfe.support_].tolist()
print("RFE selected features:", wrapper_selected)

# Show ranking
rfe_ranking = pd.Series(rfe.ranking_, index=X.columns).sort_values()
print(rfe_ranking.head(20))
```

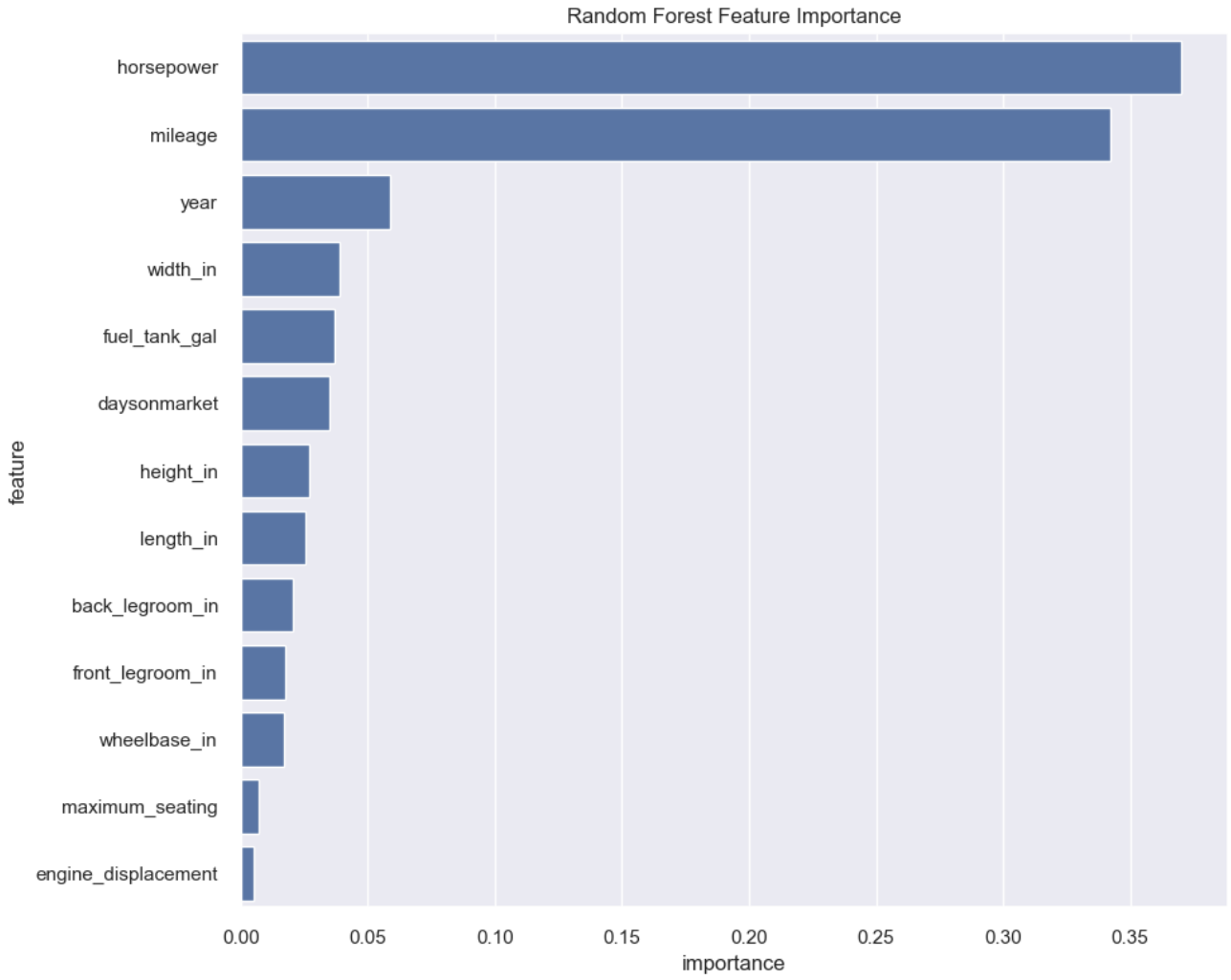
```
Fitting estimator with 34 features.
Fitting estimator with 29 features.
Fitting estimator with 24 features.
RFE selected features: ['back_legroom_in', 'daysonmarket', 'front_legroom_in',
'fuel_tank_gal', 'height_in', 'horsepower', 'mileage', 'wheelbase_in', 'width_in', 'year'
, 'age', 'horsepower_squared', 'fuel_tank_gal_squared', 'height_in_squared',
'mileage_squared', 'annual_mileage', 'horsepower/volume', 'power_per_displacement',
'volume_per_seat', 'is_awd_4wd']
back_legroom_in           1
volume_per_seat           1
power_per_displacement    1
horsepower/volume         1
annual_mileage            1
mileage_squared           1
height_in_squared         1
fuel_tank_gal_squared     1
horsepower_squared        1
age                       1
width_in                  1
year                      1
mileage                   1
horsepower                1
height_in                 1
fuel_tank_gal             1
front_legroom_in          1
daysonmarket              1
wheelbase_in              1
is_awd_4wd                1
dtype: int64
```

Embedded Method

Lasso regression shrinks the coefficients by a constant. It can also shrink some coefficients to 0, making them obvious targets to remove from the dataset. A random forest is used with the embedded feature.

```python
# Lasso Regression
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

lasso = Lasso(alpha=0.01, random_state=42)
lasso.fit(X_scaled, y)

lasso_selected = X.columns[lasso.coef_ != 0].tolist()
print("Lasso selected features:", lasso_selected)

# Random Forest Feature Importance
rf = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X, y)

importance_df = pd.DataFrame({
    'feature': X.columns,
    'importance': rf.feature_importances_
}).sort_values('importance', ascending=False)

# Plot top 20
plt.figure(figsize=(10, 8))
sns.barplot(data=importance_df.head(20), x='importance', y='feature')
plt.title('Random Forest Feature Importance')
plt.tight_layout()
plt.show()

embedded_selected = importance_df.head(20)['feature'].tolist()
```

```
Lasso selected features: ['back_legroom_in', 'daysonmarket', 'engine_displacement',
'front_legroom_in', 'fuel_tank_gal', 'height_in', 'horsepower', 'length_in',
'maximum_seating', 'mileage', 'wheelbase_in', 'width_in', 'year']
```




```python
# Find features selected by all 3 methods
all_selected = set(filter_selected) & set(wrapper_selected) & set(embedded_selected)
print("Features selected by ALL methods:", all_selected)

# Features selected by at least 2 methods
from collections import Counter
all_features = filter_selected + wrapper_selected + embedded_selected
feature_counts = Counter(all_features)
majority_selected = [f for f, count in feature_counts.items() if count >= 2]
print("Features selected by at least 2 methods:", majority_selected)

# Use majority vote as your final feature set
X_final = X[majority_selected]

# feature_cars2 is the feature_cars dataset with only the top 20 columns selected from
the script above
feature_cars2 = feature_cars[majority_selected]
quantDDA(feature_cars2)
```

```
Features selected by ALL methods: {'age', 'fuel_tank_gal', 'is_awd_4wd', 'mileage',
'horsepower/volume', 'height_in', 'horsepower', 'fuel_tank_gal_squared', 'wheelbase_in',
'width_in', 'annual_mileage', 'mileage_squared', 'horsepower_squared', 'year'}
Features selected by at least 2 methods: ['fuel_tank_gal', 'height_in', 'horsepower',
'mileage', 'wheelbase_in', 'width_in', 'year', 'age', 'horsepower_squared',
'fuel_tank_gal_squared', 'height_in_squared', 'mileage_squared', 'annual_mileage',
'fake_volume_in', 'horsepower/volume', 'is_awd_4wd', 'back_legroom_in', 'daysonmarket',
'front_legroom_in', 'power_per_displacement', 'volume_per_seat']
```

	Feature	Number of Observations	Number of Entries	Number of Unique Entries	Number of Missing Entries	Number of Outliers (IQR)	Number of Extreme Values (top /bottom 1%)	
0	fuel_tank_gal	197762	197762	109	0	2359	2513	
1	height_in	197762	197762	200	0	0	3931	
2	horsepower	197762	197762	202	0	232	2992	
3	mileage	197762	197762	60132	0	6323	1978	
4	wheelbase_in	197762	197762	155	0	82	3606	
5	width_in	197762	197762	177	0	0	3034	
6	year	197762	197762	9	0	0	0	
7	age	197762	197762	7	0	4177	0	
8	horsepower_squared	197762	197762	202	0	1519	2992	
9	fuel_tank_gal_squared	197762	197762	109	0	8003	2513	[174.239999
10	height_in_squared	197762	197762	200	0	0	3931	[4369.2099
11	mileage_squared	197762	197762	60132	0	23022	1978	
12	annual_mileage	197762	197762	71561	0	6139	1978	
13	fake_volume_in	197762	197762	1071	0	174	3488	[8
14	horsepower/volume	197762	197762	1777	0	5064	3910	[0.000195544771
15	is_awd_4wd	197762	197762	2	0	0	0	
16	back_legroom_in	197762	197762	114	0	1933	2765	
17	daysonmarket	197762	197762	183	0	11610	1781	
18	front_legroom_in	197762	197762	59	0	4705	2571	
19	power_per_displacement	197762	197762	327	0	411	2930	
20	volume_per_seat	197762	197762	1130	0	352	1815	[17

Baseline Model Comparison

First Model is pre feature engineering

Using the same technique from phase 1 to generate the baseline model, a new baseline model is generated with the feature engineering data for comparison

```
def baseline_model(df):
    # Set X as columns excluding price and vin
    X = df.drop(columns=["price", "vin", "body_type", "franchise_dealer"], errors='ignore'
)

    # Drop columns that are almost entirely unique (most likely identifiers)
    high_unique_cols = [
        col for col in X.select_dtypes(include=["object", "string"]).columns
        if X[col].nunique() / len(X) > 0.95
    ]
    X = X.drop(columns=high_unique_cols, errors="ignore")

    # Baseline simplification: keep only numeric features (avoids string errors)
    X = X.select_dtypes(include=["number"]).fillna(0)

    # Set Y as price
    y = cars_clean["price"]

    from sklearn.model_selection import train_test_split

    # 1: 70% training, 30% temporary
    X_train, X_temp, y_train, y_temp = train_test_split(
        X, y,
        test_size=0.30,
        random_state=42
    )

    # 2: split 30% into 15% validation and 15% testing
    X_val, X_test, y_val, y_test = train_test_split(
        X_temp, y_temp,
        test_size=0.50,
        random_state=42
    )

    from sklearn.linear_model import LinearRegression

    # Setting the Linear Regression
    model = LinearRegression()
    model.fit(X_train, y_train)

    import matplotlib.pyplot as plt

    # Predictions on test set
```

```
    y_pred_test = model.predict(X_test)

    plt.figure(figsize=(6,6))
    plt.scatter(y_test, y_pred_test, alpha=0.3)

    # Perfect prediction line
    plt.plot(
        [y_test.min(), y_test.max()],
        [y_test.min(), y_test.max()],
        color='red'
    )

    plt.xlabel("Actual Price")
    plt.ylabel("Predicted Price")
    plt.title("Actual vs Predicted Prices (Test Set)")
    plt.show()

    from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

    print("Train R2:", r2_score(y_train, model.predict(X_train)))
    print("Validation R2:", r2_score(y_val, model.predict(X_val)))
    print("Test R2:", r2_score(y_test, y_pred_test))

baseline_model(clean_cars)
baseline_model(feature_cars)
baseline_model(feature_cars2)
```


```
Train R2: 0.707726577842083
Validation R2: 0.7084075082975132
Test R2: 0.7046003212025849
```


```
Train R2: 0.7614258511126683
Validation R2: 0.7611538150651711
Test R2: 0.757706903960172
```


```
Train R2: 0.7441213296897466
Validation R2: 0.7441256149454307
Test R2: 0.7410454901621462
```

2.2 Comparison Discussion

Looking at the scores above, the dataset that contains all the new features scores the highest. This dataset scores roughly 6 points higher than the raw clean dataset. The partial-feature dataset (feature dataset with feature selection) only scores roughly 4 points higher than the raw clean dataset and 2 points less than the full feature dataset. However, the partial-feature dataset only contains 20 columns of data compared to the 47 columns in the feature dataset and 29 columns in the clean cars dataset.

Further discussion is found below in Phase 2.4 Interpretation

2.3 Unsupervised Learning

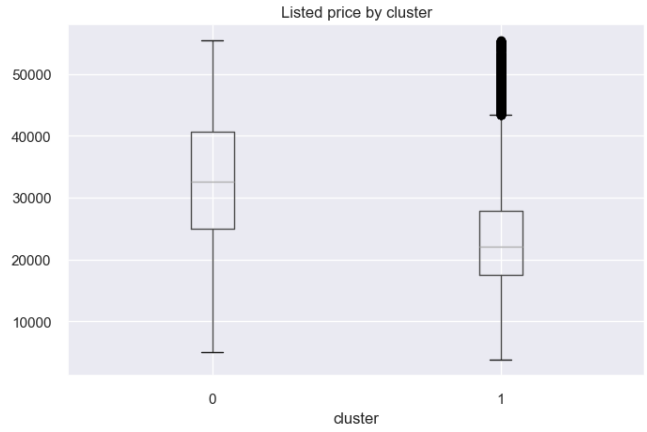
We address **Research Question 2** (from the EDA section): can we find **natural groupings** of vehicles in feature space without using price as a clustering input? We **exclude** ***price*** from the clustering features so clusters reflect specifications and market attributes, then **compare** ***price*** **across clusters** to interpret segments (e.g. lower vs higher priced groups).

Method: We standardize numeric features, run **K-Means** for several values of *k*, and pick *k* using the **silhouette score** (computed on a random subsample for speed). We visualize structure with **PCA** (2D) and summarize **price** by cluster with boxplots and median/mean tables.

```
# K-Means clustering + PCA (§2.3). Uses engineered features if available.
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

df_ul = globals().get("feature_cars")
if df_ul is None:
    df_ul = globals().get("clean_cars")
if df_ul is None:
    raise NameError("Run Phase 2 to load clean_cars and build feature_cars.")

# Cap rows for tractable K-Means / silhouette (full dataset is large)
max_rows = 50_000
df_work = df_ul.sample(n=min(max_rows, len(df_ul)), random_state=42).copy()

price_series = df_work["price"]
drop_for_cluster = {"price", "vin", "listed_date"}
X_cluster = df_work.drop(columns=[c for c in drop_for_cluster if c in df_work.columns],
errors="ignore")
X_cluster = X_cluster.select_dtypes(include=["number"]).fillna(0)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_cluster)

# Silhouette on a fixed subsample (faster than all-pairs on 50k points)
rng = np.random.RandomState(42)
sil_idx = rng.choice(len(X_scaled), size=min(8_000, len(X_scaled)), replace=False)

silhouette_by_k = {}
for k in range(2, 9):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels_k = km.fit_predict(X_scaled)
    silhouette_by_k[k] = silhouette_score(X_scaled[sil_idx], labels_k[sil_idx])

best_k = max(silhouette_by_k, key=silhouette_by_k.get)
print("Silhouette (subsample) by k:", {k: round(silhouette_by_k[k], 4) for k in
silhouette_by_k})
print("Chosen k:", best_k)
```

```
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
cluster_id = kmeans.fit_predict(X_scaled)

pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)

plot_idx = rng.choice(len(X_scaled), size=min(15_000, len(X_scaled)), replace=False)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sc = axes[0].scatter(
    X_pca[plot_idx, 0], X_pca[plot_idx, 1],
    c=cluster_id[plot_idx], cmap="tab10", alpha=0.35, s=8,
)
axes[0].set_title("PCA of listings (K-Means clusters)")
axes[0].set_xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% var)")
axes[0].set_ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% var)")
plt.colorbar(sc, ax=axes[0], label="cluster")

summary_df = pd.DataFrame({"price": price_series.values, "cluster": cluster_id})
summary_df.boxplot(column="price", by="cluster", ax=axes[1])
axes[1].set_title("Listed price by cluster")
axes[1].set_xlabel("cluster")
plt.suptitle("")
plt.tight_layout()
plt.show()

table = summary_df.groupby("cluster")["price"].agg(["count", "median", "mean"]).
sort_values("median")
print("\nPrice by cluster:")
print(table)
```

```
Silhouette (subsample) by k: {2: 0.2729, 3: 0.1754, 4: 0.1489, 5: 0.1339, 6: 0.1392, 7:
0.1398, 8: 0.1378}
Chosen k: 2
```




```
Price by cluster:
         count   median          mean
cluster
1        35326  22098.5  23338.707851
0        14674  32520.0  32790.850595
```

Interpretation. Clusters are formed only from numeric vehicle/listing features (price held out), so separation in PCA space reflects combinations of specs and market fields rather than price itself. If median price differs clearly across clusters, those groups behave like interpretable **market segments** (e.g. lower vs higher price bands for different feature mixes). Silhouette-guided *k* balances cohesion and separation; segments are descriptive, not a predictive model of price.

2.4 Interpretation

Feature Importance

feature_cars2 features the 20 most important features for price prediction. This dataset does not offer the best performance. feature_cars offers slightly better performance as seen in 2.2 Comparison Discussion. If there are computational constraints, it is better to use feature_cars2 because if offers very good performance for only 20 columns of data. If there are no computational constraints, use feature_cars.

horsepower and mileage are very good predictors of price. These two variables constituency place as top predictors. The variables year and age also appear to be strong predictors of price. Although they aren't as good as horsepower and mileage, they should not be ignored. It's expected that horsepower would be a strong predictor because this is the primary indicator of a car's performance. More horsepower means that car has better acceleration, top speed and towing capacity, all of which typically demand a premium price for high performance. mileage is also a strong predictor of price because it signifies how much the car has been used. Typically, cars are made to drive a certain amount of mileage before breaking, so high mileage indicates how close the car is to being worth scraps. Along with mileage, year and age are also indicators of a vehicle's age and how much longer the car might last.

Related to the Advanced Supervised Models , the feature importance analysis for the Random Forest and XGBoost Models reveal that horsepower, mileage, and vehicle year are the most influential variables for predicting price.

Horsepower is the most important feature; this indicates that vehicle performance plays a major role in determining the price. We also notice that mileage is highly influential, and that higher mileage is associated to lower prices (we can assumer due to vehicle wear and/or depreciation). The vehicle year is the other key factor, which reflect the impact of age on price.

The feature importance results are consistent across both models, which strengthens the reliability of the findings and confirm that both usage-related and performance-related characteristics influence the vehicle price. Other feature such as vehicle dimensions and fuel attributes have a smaller but still noticeable impact.

Here are some important engineered features:

- is_awd_4wd ranked in all 3 tests. This feature checks if the vehicle is all-wheel drive or 4-wheel drive. This feature was engineered to hold true or false values (1 or 0) so the machine learning algorithms have numerical values to work with.
- horsepower/volume ranked in all 3 tests. It doesn't score the highest, but it is consitent and provides good insight on a car's predicted price. It is a feature that calculates how much horsepower there is per unit of volume (inches).
- fake_volume_in ranked in all 3 tests. It had very mixed results in all three tests, ranking high in some and low in others. This variable does not contain the actual vehicles volume data, instead it simply finds the fake volume of the vehicle by calculating its height * width * length.
- power_per_displacement, volume_per_seat and fuel_per_volume. All three of these features barely make the cut into feature_cars2 and only appear significant in 2 of 3 algorithms. These engineered features alone don't provide much value, but they are still significant enough to be included in the top 20 features.

Relation to Research Question

The results seem to directly answer the research question by demonstrating that vehicle specifications, usage history, and dealership-related characteristics can predict the selling price of a used vehicle. Particularly, the strong influence of mileage, horsepower, and vehicle age confirms that both usage and performance features are key determinants of price. The use of advanced models such as Random Forest and XGBoost further shows that capturing relationships that are non-linear significantly improves predictive performance.

Partial Dependence Plots and Insights

The partial dependence values had to follow a log distribution because the original values do not provide any meaningful insight. Because of this, the graphs do not show actual prices. However, we can still analyze the change compared to relative pricing.

The first and fourth graph that focus on year and age. These two graphs are observed together because they share similar results, but the primary focus will be on graph 1 (year). We see that the price of the car increases almost linearly with increasing years. However, the plot is not perfectly linear. Between the years 2017 and 2019 we see a smaller increase in price compared to the rest of the graph. This phenomenon can also be observed at the lowest and highest years of this graph. With this observation we see that cars hold their value well for the first year then drop significantly after that. They once again hold there value for a few more years before seeing another significant drop. Cars made before 2014 don't lose value as fast anymore compared to newer cars.

The second graph focuses on mileage. We observe that cars lose value very slowly for the first few thousand miles driven. At one point (after roughly 8000 of miles are driven) a steep decline in car value is observed. After that initial steep decline, the car's value decrease slowly but steadily until about 58,000 miles have been driven. At this point another small but quick decline is observed, but quickly corrects itself back to the same previous slow steady decline. This graphs shows that cars tend to hold their value well for a few thousand miles, but then suddenly lose value once a certain point is reached. But after this huge drop in value, cars begin to depreciate in value slowly and constistenly.

The third graph shows how mileage effects price. The graph shows no increase in price until a little under 100 horsepower. After this point that graph increases like a staircase until about 200 horspower. At the 200 horsepower marker, there is a huge leap in price. However, after this huge leep in price the graph once again plateaus and only sees small increases in price. This graph shows that at about 100 horsepower, the demand more small increases in horspower drastically increase the value of a vehicle. The asking price for a vehicle with a little over 200 horsepower is worth significantly more than a vehicle with even a bit less than 200 horsepower. After a little over 200 horsepower is reached, the price of a vehicle slowly increases compared to the increases seen between 100 and 200 horsepower.

```
import numpy as np
from sklearn.inspection import PartialDependenceDisplay

y_log = np.log1p(y)  # log(price + 1)

rf = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X_final, y_log)

# Then replot PDPs
PartialDependenceDisplay.from_estimator(rf, X_final, features=['year', 'mileage',
'horsepower', 'age'])
plt.show()
```

```
/Users/charlottelauzon/miniconda3/envs/COMP333_PROJECT_GROUP_O/lib/python3.13/site-packages/sklearn/inspection/_partial_dependence.py:721: FutureWarning: The column 6 contains integer data. Partial dependence plots are not supported for integer data: this can lead to implicit rounding with NumPy arrays or even errors with newer pandas versions. Please convert numerical featuresto floating point dtypes ahead of time to avoid problems. This will raise ValueError in scikit-learn 1.9.
  warnings.warn(
/Users/charlottelauzon/miniconda3/envs/COMP333_PROJECT_GROUP_O/lib/python3.13/site-packages/sklearn/inspection/_partial_dependence.py:721: FutureWarning: The column 7 contains integer data. Partial dependence plots are not supported for integer data: this can lead to implicit rounding with NumPy arrays or even errors with newer pandas versions. Please convert numerical featuresto floating point dtypes ahead of time to avoid problems. This will raise ValueError in scikit-learn 1.9.
  warnings.warn(
```